

---

# Learning Decentralized LLM Collaboration with Multi-Agent Actor Critic

---

Shuo Liu<sup>1</sup> Tianle Chen<sup>1</sup> Ryan Amiri<sup>1</sup> Christopher Amato<sup>1</sup>

## Abstract

Recent work has explored optimizing LLM collaboration through Multi-Agent Reinforcement Learning (MARL). However, most MARL fine-tuning approaches rely on predefined execution protocols, which often require centralized execution. Decentralized LLM collaboration is more appealing in practice, as agents can run inference in parallel with flexible deployments. Also, current approaches use Monte Carlo methods for fine-tuning, which suffer from high variance and thus require more samples to train effectively. Actor-critic methods are prevalent in MARL for dealing with these issues; thus, we developed Multi-Agent Actor-Critic (MAAC) methods to optimize decentralized LLM collaboration. In this paper, we analyze when and why these MAAC methods are beneficial. We propose 2 MAAC approaches, **CoLLM-CC** with a Centralized Critic and **CoLLM-DC** with Decentralized Critics. Our experiments across writing, coding, and game-playing domains show that Monte Carlo methods and CoLLM-DC can achieve performance comparable to CoLLM-CC in short-horizon and dense-reward settings. However, they both underperform CoLLM-CC on long-horizon or sparse-reward tasks, where Monte Carlo methods require substantially more samples and CoLLM-DC struggles to converge. Our code is available at <https://github.com/OpenMLRL/CoMLRL/releases/tag/v1.3.6>.

## 1. Introduction

Advanced LLMs have demonstrated remarkable capabilities in natural language understanding and generation (Achiam et al., 2023; Anil et al., 2023; Bai et al., 2023; Guo et al., 2025). This progress has driven growing efforts to transform

them into autonomous agents (Yao et al., 2022; Liu et al., 2023; Yang et al., 2024).

In this context, it is becoming popular to explore coordinating multiple LLMs to improve performance where agents are specified by roles, e.g., generators, planners, or verifiers (Wu et al., 2023; Du et al., 2023; Skreta et al., 2023; Qian et al., 2024; Zhang et al., 2025a; Wang et al., 2026a). Building on the studies of Multi-Agent Systems (MAS) (Weiss, 1999; Stone & Veloso, 2000; Van der Hoek & Wooldridge, 2008; Shoham & Leyton-Brown, 2009), recent methods employ Multi-Agent Reinforcement Learning (MARL) to optimize their interactions (Liu et al., 2025a;b; Feng et al., 2025; Chen et al., 2025; Subramaniam et al., 2025; Wu et al., 2025b; Chen et al., 2026; Feng et al., 2026). However, most existing approaches remain confined to predefined execution paradigms, which limits their applicability to broader settings. Moreover, these agents often rely on extensive inter-agent communication to accomplish tasks, requiring centralized execution, which results in limited scalability as well as potential privacy issues in larger-scale MAS.

Decentralized systems have been studied for decades, where each agent is deployed separately and executes independently based on its own observations (Waldo et al., 1996; Ghosh, 2006; Oliehoek & Amato, 2016; Amato, 2024). Leveraging decentralized LLMs to complete tasks is beneficial, as it reduces memory and storage pressure on each node and improves the efficiency (Huang et al., 2019; Lepikhin et al., 2020; Douillard et al., 2023; Wu et al., 2025a). However, how to effectively optimize these decentralized LLM agents to collaborate remains an open question.

Although Monte Carlo methods are widely adopted in RL fine-tuning due to their simplicity and efficiency (Li et al., 2023; Ahmadian et al., 2024; Shao et al., 2024; Guo et al., 2025; Hu et al., 2025; Liu et al., 2026d), extending them to optimize multi-LLM collaboration faces many difficulties (Liao et al., 2025; Zhang et al., 2025a; Hong et al., 2025; Liu et al., 2026c). Agents need to wait until the end of an episode to receive return signals with high variance. This leads to poor sample efficiency and limits practicality in long-horizon or episodic tasks (Sutton & Barto, 1998).

In this paper, we develop Multi-Agent Actor-critic (MAAC) methods for optimizing decentralized LLM collaboration. We analyze when and why MAAC methods are beneficial

---

<sup>1</sup>Northeastern University, Boston, MA. Correspondence to: Christopher Amato <c.amato@northeastern.edu>.

for MARL fine-tuning and introduce 2 approaches, **CoLLM-CC** that employs a centralized critic to estimate joint history values, and **CoLLM-DC** that uses decentralized critics to estimate individual history values. Our evaluation across writing, coding, and game-playing domains shows that, in dense-reward and short-horizon writing tasks, Monte Carlo methods and CoLLM-DC achieve performance comparable to CoLLM-CC; while in sparse-reward coding tasks and long-horizon Minecraft tasks, both underperform CoLLM-CC, where Monte Carlo methods require substantially more samples for training, and CoLLM-DC fails to converge.

Our contributions are summarized as follows: (i) We develop MAAC methods to optimize decentralized LLM collaboration and analyze their advantages; (ii) We propose CoLLM-DC and CoLLM-CC as 2 MAAC approaches; and (iii) Our results on collaborative writing, coding, and game-playing tasks demonstrate that CoLLM-CC consistently outperforms Monte Carlo methods and CoLLM-DC, particularly in long-horizon tasks with sparse rewards.

## 2. Related Works

### 2.1. LLM Collaboration

A lot of works employ LLM agents to collaborate with complementary roles (e.g., planner, retriever, generator, verifier) (Wu et al., 2023; Du et al., 2023; Skreta et al., 2023; Qian et al., 2024; Zhang et al., 2025a; Liu et al., 2025a;b; Wu et al., 2025b; Chen et al., 2025; Zhao et al., 2025a;b; Feng et al., 2026; Chen et al., 2026). While such role specialization can improve performance, most frameworks enforce rigid protocols, such as refinement pipelines, hierarchical decompositions, or discussion loops, which are hand-coded, entail heavy communication overhead, and typically require centralized execution to produce a solution. Moreover, it is still unclear whether such protocols are optimal cooperation schemes in general (Estornell & Liu, 2024; Cemri et al., 2025; Zhang et al., 2025b). An alternative is decentralized LLM collaboration, where agents run inference in parallel under limited or no communication (Chen et al., 2024; Qi et al., 2025; Yang et al., 2025a; Liu et al., 2026c). This paradigm improves efficiency and enables more flexible deployment across multiple small LLMs. However, how to effectively optimize such decentralized collaboration remains underexplored.

### 2.2. Actor-Critic Methods in Cooperative MARL

Actor-Critic (AC) is a widely used policy-gradient architecture in which an actor model selects actions based on its policy, and a critic model estimates values during training (Sutton & Barto, 1998; Konda & Tsitsiklis, 1999). In the decentralized multi-agent learning setting, each agent acts based on its local observation (Oliehoek & Amato, 2016;

Albrecht et al., 2024), and the critic can be instantiated either as an individual critic for independent learning (De Witt et al., 2020; Lee et al., 2022), or as a centralized critic that conditions on joint history (Lowe et al., 2017; Foerster et al., 2018; Lyu et al., 2021; Yu et al., 2022; Lyu et al., 2023). The centralized-critic methods can also leverage additional information during centralized training to support more accurate and stable gradient estimates, while agents still execute in a decentralized manner (CTDE).

## 3. Background

### 3.1. Decentralized LLM Collaboration

In decentralized LLM collaboration, multiple LLM agents cooperate to solve a class of tasks. Each task is expressed with natural-language observations and provided to the agents as individual prompts. Agents then produce responses in parallel according to their own policies conditioned on the prompt. The responses of all agents are aggregated to form a solution. All agents share a joint reward based on the aggregated outcome.

After each turn, users, external tools, and other models validate the proposed solution and provide new requirements, suggestions, or constraints for the next iteration. This feedback, together with the dialog history, is maintained and serves as both a decision-making context and an experience for training. The interaction repeats until the task is solved or a predefined turn limit is reached. The goal of agents is to achieve higher-quality solutions with minimal iterations.

Decentralized LLM collaboration offers great advantages by allowing specialized agents to divide and conquer complex problems and operate in parallel to improve efficiency. Rather than relying on a single gigantic LLM in a centralized system, lightweight agents in a decentralized system can focus on subtasks guided by their own prompts, thereby greatly reducing the overhead of maintaining long contexts and extracting relevant semantics. Deploying smaller models locally is also typically easier, safer, and more scalable, making decentralized LLM collaboration well-suited for applications such as long article generation, software engineering, and embodied coordination (Gao et al., 2023; Chen et al., 2024; Wu et al., 2025a; Liu et al., 2026a;b;c; Hao et al., 2025; Wang et al., 2026b).

### 3.2. LLM Dec-POMDP

Decentralized LLM collaboration can be seen as a subclass of the Dec-POMDP (Oliehoek & Amato, 2016), which is denoted as  $\langle \mathcal{I}, \mathcal{V}, \mathcal{C}, \mathcal{M}, \mathcal{S}, \{\mathcal{O}_i\}, \{\mathcal{A}_i\}, R, T, H \rangle$ .

In an LLM Dec-POMDP,  $\mathcal{I}$  denotes a set of  $n$  LLM agents,  $\mathcal{V}$  is the token vocabulary in LLM inputs and outputs,  $\mathcal{C}$  is the input context window size for each agent, and  $\mathcal{M}$

is the max number of tokens in each LLM output.  $\mathcal{S} : \mathcal{S}^{\text{sys}} \times \mathcal{S}^{\text{usr}}$  denotes the full global state space. At turn  $t$ ,  $s_t \in \mathcal{S}$  consists of accessible parts  $s_t^{\text{sys}} \in \mathcal{S}^{\text{sys}}$  from the system and inaccessible  $s_t^{\text{usr}} \in \mathcal{S}^{\text{usr}}$  parts from users. In RL fine-tuning via verifiable rewards (RLVR), only the accessible parts are used by the reward model.  $\mathcal{O}_i$  is the observation space for agent  $i$  with  $\mathcal{O} = \times_i \mathcal{O}_i$  the joint observation space, where a local observation  $o_{i,t}$  is a natural language prompt providing a partial and noisy view of  $s_t$ , and  $|\mathcal{O}_i| = |\mathcal{V}|^C$ .  $\mathcal{A}_i$  is the action space for agent  $i$  with  $\mathcal{A} = \times_i \mathcal{A}_i$  the joint action space, where an individual action  $a_{i,t}$  is its response to the given prompt, and  $|\mathcal{A}_i| = |\mathcal{V}|^M$ . The joint reward function  $R : \mathcal{S}^{\text{sys}} \times \mathcal{A} \rightarrow \mathbb{R}$  is implemented via predefined rules or a pretrained reward model. At turn  $t$ , the joint rewards  $r_t \leftarrow R(s_t^{\text{sys}}, \mathbf{a}_t)$  are determined by the accessible part of current state  $s_t^{\text{sys}}$  and the agents' joint action  $\mathbf{a}_t = \{a_{1,t}, \dots, a_{n,t}\}$ .  $T : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$  is the underlying stochastic state transition function. At turn  $t$ , the agents' joint actions  $\mathbf{a}_t$  induce a shift to a new state  $s_{t+1} \sim T(\cdot | s_t, \mathbf{a}_t)$ , which reflects the updates in the user state and the states of external models and systems.  $H$  is the episode horizon, the turn limit of a dialog.

Since both the accessible  $s^{\text{sys}}$  and the inaccessible  $s^{\text{usr}}$  parts of the states cannot be directly observed by the agents. Each agent maintains its local observation-action history  $h_{i,t} = \{o_{i,0}, a_{i,0}, \dots, o_{i,t}\}$  to infer information about the state  $s_t = \{s_t^{\text{sys}}, s_t^{\text{usr}}\}$ . The history of all agents forms a joint history  $\mathbf{h}_t = \{h_{1,t}, \dots, h_{n,t}\}$ , and all agents' policies forms a joint policy  $\pi = \{\pi_1, \dots, \pi_n\}$ . A solution to a Dec-POMDP is a joint policy that maximizes the expected cumulative reward over the horizon  $H$ ,  $\pi^* = \{\pi_1^*, \dots, \pi_n^*\} = \arg \max_{\pi} \mathbb{E}_{\pi} \left[ \sum_{t=0}^{H-1} r_t \right]$ . We describe 3 LLM Dec-POMDP instantiations in Section 6.

## 4. Cooperative MARL for LLM Fine-Tuning

In this section, we introduce cooperative MARL methods and provide a theoretical analysis of their advantages and limitations when applied to LLM fine-tuning.

### 4.1. MA-REINFORCE

Multi-Agent REINFORCE (MA-REINFORCE) is a class of multi-agent policy gradient methods in which each agent  $i$  maintains an individual policy  $\pi_{\theta_i}$ , and updates it according to Monte-Carlo estimates of joint returns (Peshkin et al., 2001). Mathematically,

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{\pi} \left[ \sum_{t=0}^{H-1} \rho_{i,t} \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t} | h_{i,t}) (G(\mathbf{h}_t) - b(\mathbf{h}_t)) \right], \quad (1)$$

where  $G(\mathbf{h}_t) = \mathbb{E}_{\pi} \left[ \sum_{\tau=t}^{H-1} \gamma^{\tau-t} r_{\tau} | \mathbf{h}_t \right]$  is the expected return from  $\mathbf{h}_t$ ,  $\rho_{i,t} = \frac{\pi_{\theta_i}(a_{i,t} | h_{i,t})}{\pi_{\theta_i, \text{old}}(a_{i,t} | h_{i,t})}$  is the importance sampling ratio for correcting the off-policy action sampled,

and  $b(\mathbf{h}_t)$  denotes an action-independent baseline. In MA-REINFORCE, agents learn directly from sampled returns without a critic. However, this method can not scale well to long-horizon online learning, because return signals are only available at dialog termination and Monte Carlo estimates suffer from high variance due to accumulated stochasticity.

MA-REINFORCE is initially designed to take one action at each turn during rollout. Recent MARL fine-tuning methods aim to reduce the variance of gradient estimation by generating  $K$  i.i.d. joint actions  $\{\mathbf{a}_t^1, \dots, \mathbf{a}_t^K\}$  as a group for each history  $\mathbf{h}_t$  following policy  $\pi(\cdot | \mathbf{h}_t)$  (Zhang et al., 2025a; Hong et al., 2025; Liu et al., 2026c; Feng et al., 2026). In the multi-turn environment, each action yields a successor observation  $\{\mathbf{o}_{t+1}^1, \dots, \mathbf{o}_{t+1}^K\}$ , and obtaining lower-variance gradients for these successor histories  $\{\mathbf{h}_{t+1}^1, \dots, \mathbf{h}_{t+1}^K\}$  also requires  $K$  samples. This recursively expands into a  $K$ -ary rollout tree of depth  $H$  (Yang et al., 2025b; Ding & Ye, 2025; Ji et al., 2025; Li et al., 2025).

We show that the average gradient estimator by  $K$ -sampling MA-REINFORCE is unbiased in Proposition 4.1.

**Proposition 4.1.** *For each history  $\mathbf{h}_t$  at  $t$ ,  $t \in [0, H)$ , suppose agents sample  $K \geq 1$  i.i.d. joint actions  $\{\mathbf{a}_t^1, \dots, \mathbf{a}_t^K\}$  from  $\pi(\cdot | \mathbf{h}_t)$ . For each  $\mathbf{a}_t^k$ , an independent  $K$ -ary rollout tree is produced to estimate the corresponding expected Monte Carlo return  $G_t^k$ . For agent  $i$ , define  $\bar{g}_{i,t}$  as the averaged gradient estimate over  $\{a_{i,t}^1, \dots, a_{i,t}^K\}$ ,*

$$\bar{g}_{i,t} = \frac{1}{K} \sum_{k=1}^K \rho_{i,t}^k \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t}^k | h_{i,t}) G_t^k.$$

*Then  $\bar{g}_{i,t}$  is unbiased for the true gradient  $g_{i,t}^*$ ,*

$$\mathbb{E}_{\pi}[\bar{g}_{i,t} | h_{i,t}] = \mathbb{E}_{\pi}[g_{i,t}^* | h_{i,t}].$$

*Proof sketch.* The gradient estimates of  $K$ -sampling MA-REINFORCE equal those of the MA-REINFORCE in (Peshkin et al., 2001). By Theorem 1 therein,  $\bar{g}_{i,t}$  is unbiased. A full proof is provided in Appendix A.  $\square$

Proposition 4.2 measures the variance reduction of  $K$ -sampling MA-REINFORCE. Under an independent rollout assumption, its variance scales as  $1/K^{H-t}$  in  $K$ .

**Proposition 4.2.** *Under the setting of Proposition 4.1, for any history  $h_{i,t}$ , we assume the gradient estimates for  $\{a_{i,t}^1, \dots, a_{i,t}^K\}$  are independent and episodes are not terminated early. Let  $\sigma^2$  be the gradient estimate variance when  $K = 1$ , the variance of  $\bar{g}_{i,t}$  satisfies,*

$$\text{Var}_{\pi}(\bar{g}_{i,t} | h_{i,t}) = \frac{\sigma^2}{K^{H-t}}.$$

*Proof sketch.* Without early termination, the  $K$ -ary rollout tree has  $K^{H-t}$  i.i.d. leaf rollouts from  $h_{i,t}$ . Under independent gradient estimation assumption, all these samples are

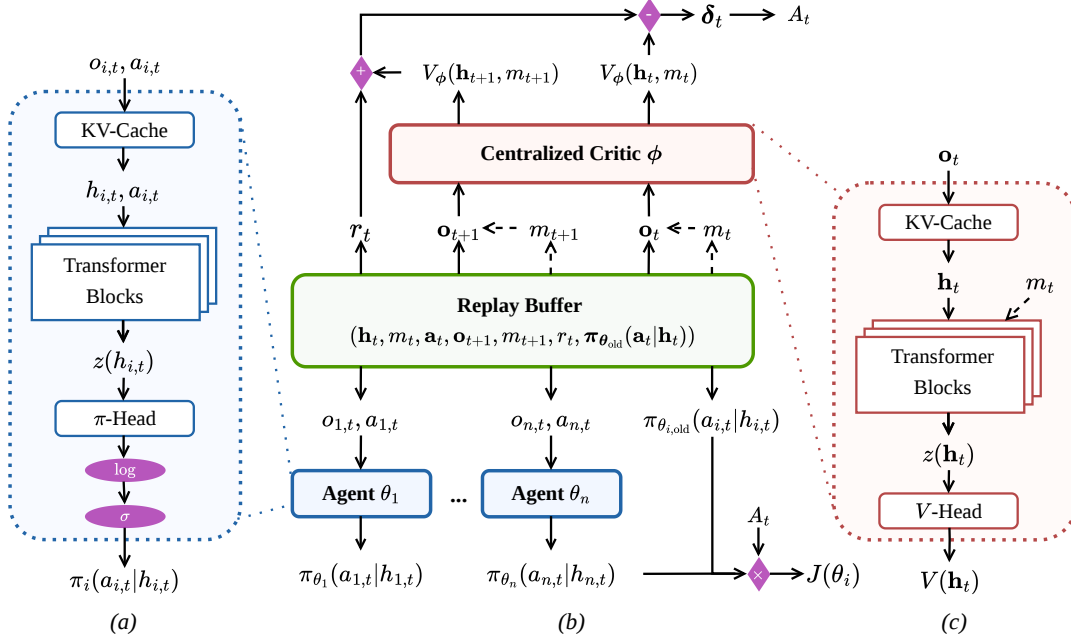


Figure 1. Illustration of CoLLM-CC framework: (a) The agent model structure; (b) The overall centralized-critic architecture; (c) The critic model structure. The corresponding CoLLM-DC framework is shown in Appendix B.

effectively used in  $\bar{g}_{i,t}$ , yielding  $\text{Var}(\bar{g}_{i,t}|h_{i,t}) = \sigma^2/K^{H-t}$ . We provide the formal justification in Appendix A.  $\square$

However, this variance reduction comes at the cost of a rapidly increasing LLM inference calls to maintain the rollout tree, as shown in Proposition 4.3.

**Proposition 4.3.** *Consider an  $H$ -horizon episode without early termination  $t \in [0, H)$ . Suppose MA-REINFORCE expands a full  $K$ -ary rollout tree ( $K \geq 1$ ) and, at each history node, draws  $K$  i.i.d. joint actions. Each LLM runs one inference at a time to produce a response. Then the total number of inference calls required for this episode is  $N_{\text{call}}(n, K, H) = \frac{nK(K^H-1)}{K-1}$ .*

*Proof.* Since the rollout forms a  $K$ -ary tree in each episode, the total number of actions forms a geometric series. Thus,  $N_{\text{call}}(n, K, H) = nK \sum_{\ell=0}^{H-1} K^\ell = \frac{nK(K^H-1)}{K-1}$ .  $\square$

## 4.2. Multi-Agent Actor-Critic

To reduce the variance of gradient estimates and improve sample efficiency, Actor-Critic (AC) methods learn a policy model (actor)  $\pi_\theta$  and a value model (critic)  $V_\phi$  (or  $Q_\psi$ ). In the multi-agent setting, AC methods are common with Decentralized Critics (Foerster et al., 2018; De Witt et al., 2020) or with a Centralized Critic (CC) (Lowe et al., 2017; Foerster et al., 2018; Yu et al., 2022).

In DC methods, each agent  $i$  maintains a local critic  $V_{\phi_i}(h_{i,t})$  and updates its policy  $\pi_{\theta_i}(\cdot|h_{i,t})$  via

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{\pi} \left[ \sum_{t=0}^{H-1} \rho_{i,t} \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t} | h_{i,t}) \delta_{i,t} \right], \quad (2)$$

where  $\delta_{i,t} = r_t + \gamma V_{\phi_i}(h_{i,t+1}) - V_{\phi_i}(h_{i,t})$ , and each critic  $V_{\phi_i}$  is updated by minimizing its TD loss,

$$\mathcal{L}(\phi_i) = \mathbb{E}_{\pi} \left[ \sum_{t=0}^{H-1} (r_t + \gamma V_{\phi_i}(h_{i,t+1}) - V_{\phi_i}(h_{i,t}))^2 \right]. \quad (3)$$

CC learns a shared value function  $V_\phi(\mathbf{h}_t)$  that conditions on the joint history  $\mathbf{h}_t$  (and even other available information during training) to update each agent’s policy  $\pi_{\theta_i}(\cdot|h_{i,t})$ ,

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{\pi} \left[ \sum_{t=0}^{H-1} \rho_{i,t} \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t} | h_{i,t}) \delta_t \right], \quad (4)$$

where  $\delta_t = r_t + \gamma V_\phi(\mathbf{h}_{t+1}) - V_\phi(\mathbf{h}_t)$ , and

$$\mathcal{L}(\phi) = \mathbb{E}_{\pi} \left[ \sum_{t=0}^{H-1} (r_t + \gamma V_\phi(\mathbf{h}_{t+1}) - V_\phi(\mathbf{h}_t))^2 \right]. \quad (5)$$

Since the critic is just a training construct, it can be removed during execution. As a result, in both DC and CC, agents can still execute in a decentralized manner.

Assuming convergence of critics, both the gradient estimates of DC in Equation 2 and CC in Equation 4 are unbiased (Lyu et al., 2021; 2023). Though this assumption can be harder to satisfy for DC methods, because each individual critic  $V_{\phi_i}$  conditions only on its local history  $h_{i,t}$ , while the policies of the other agents,  $\pi_{\theta_{-i}}(\cdot|h_{-i,t})$ , change during training and thus induce non-stationarity.

## 5. CoLLM-CC

We introduce CoLLM-CC as a representative MAAC approach and discuss key design features, with its DC counterpart, CoLLM-DC, presented in Appendix B.

### 5.1. History Representation

In cooperative MARL, each agent  $i$  typically uses a recurrent neural network (RNN) to encode its history  $h_{i,t}$  by taking  $o_{i,t}$  as input, and its hidden state  $z_{i,t}$  serving as a compact but lossy representation (Suneahg et al., 2017; Foerster et al., 2018; Rashid et al., 2020; Wang et al., 2021; Baisero et al., 2025). However, this paradigm cannot effectively scale to LLMs that take long prompt sequences as input, and language representations are often high-dimensional.

Transformers can capture long-range dependencies via attention mechanisms (Vaswani et al., 2017). This makes them well-suited for modeling dialog and interaction histories and serving as the backbone of modern LLMs. In CoLLM-CC, the dialog history is maintained in a key-value (KV) cache. At each turn, for each agent, we concatenate the KV pairs from previous turns with the new prompt from the environment (line 5 of Alg. 1), and then maintain only the most recent  $C$  pairs in the cache. The agents' KV-caches are independent and private to maintain decentralized inference. In addition to  $\mathbf{h}_t$ , the global information  $m_t$  can also be incorporated into value estimation to facilitate CC learning a richer or more accurate semantic representation during training. Such global information includes model specifications, task completion progress, or external models or tools existing in the system, which can either be appended as prompts or concatenated with  $z(\mathbf{h}_t)$  in a numerical representation.

### 5.2. Sequences as Actions

Since the human language is highly structured, rewards in many tasks are naturally defined at the level of complete responses. Considering each token an action may lead to uninformative credit assignment. Sequences can be used as macro-actions in RL fine-tuning. However, unlike traditional MARL settings with a small action space and action probabilities are readily available, the response-level action space is combinatorially large  $|\mathcal{V}|^M$ , making the probability of an entire sequence non-trivial to obtain.

CoLLM-CC employs Teacher-Forced (TF) forward passes

---

#### Algorithm 1: CoLLM-CC

---

```

1: Input: Taskset  $\mathcal{D}$ , LLM agents  $\{\pi_{\theta_i}\}_{i \in \mathcal{I}}$ , centralized
   LLM critic  $V_{\phi}$ , learning rates  $\alpha_{\pi}, \alpha_V$ , discount  $\gamma$ , hori-
   zon  $H$ , replay buffer  $\mathcal{B}$ , training epochs  $E$ 
2: for each episode do
3:   Sample a task from  $\mathcal{D}$ 
4:   Initialize prompts  $o_{i,0}$ , and  $\mathbf{o}_0 \leftarrow \{o_{i,0}\}_{i=1}^n$ 
5:   Initialize the dialog history  $h_{i,0} \leftarrow \{o_{i,0}\}$ ,  $\mathbf{h}_0 \leftarrow \{h_{i,0}\}_{i=1}^n$ , and obtain global information  $m_0$ 
6:   Initialize replay buffer  $\mathcal{B} \leftarrow \emptyset$ 
7:   for turn  $t = 0$  to  $H - 1$  do
8:     Generate response  $a_{i,t} \sim \pi_{\theta_i}(\cdot|h_{i,t})$ , and calculate
       its policy  $\pi_{\theta_i}(a_{i,t}|h_{i,t})$  with TF,  $\forall i \in \mathcal{I}$ 
9:      $\mathbf{a}_t \leftarrow \{a_{i,t}\}_{i=1}^n$ ,  $\boldsymbol{\pi}_{\theta} \leftarrow \{\pi_{\theta_i}\}_{i=1}^n$ 
10:    Record behavior policy  $\boldsymbol{\pi}_{\theta_{\text{old}}} \leftarrow \boldsymbol{\pi}_{\theta}$ 
11:    Obtain joint reward  $r_t$ , next turn prompts  $o_{i,t+1}$ ,
        $\forall i \in \mathcal{I}$ , and global information  $m_{t+1}$ 
12:    Update  $h_{i,t+1} \leftarrow \{h_{i,t}, a_{i,t}, o_{i,t+1}\}$ ,  $\forall i \in \mathcal{I}$ 
13:     $\mathbf{o}_{t+1} \leftarrow \{o_{i,t+1}\}_{i=1}^n$ ,  $\mathbf{h}_{t+1} \leftarrow \{h_{i,t+1}\}_{i=1}^n$ 
14:    Store  $(\mathbf{h}_t, m_t, \mathbf{a}_t, r_t, \mathbf{o}_{t+1}, m_{t+1}, \boldsymbol{\pi}_{\theta_{\text{old}}})$  into  $\mathcal{B}$ 
15:  end for
16:  for training epoch  $e = 1, \dots, E$  do
17:    Sample a minibatch  $\beta$  from  $\mathcal{B}$ 
18:    for each  $(\mathbf{h}_t^b, m_t^b, \mathbf{a}_t^b, r_t^b, \mathbf{o}_{t+1}^b, m_{t+1}^b, \boldsymbol{\pi}_{\theta_{\text{old}}}^b) \in \beta$  do
19:      Calculate  $\mathcal{L}^b(\phi, m^b)$  (Eq. 5)
20:      Calculate  $\pi_{\theta_i}(a_{i,t}^b|h_{i,t}^b)$  with TF,  $\forall i \in \mathcal{I}$ 
21:      Calculate  $\nabla_{\theta_i} J^b(\theta_i)$  (Eq. 4),  $\forall i \in \mathcal{I}$ 
22:    end for
23:    Update critic  $\phi \leftarrow \phi - \alpha_V \frac{1}{|\beta|} \sum_b \nabla_{\phi} \mathcal{L}^b(\phi, m^b)$ 
24:    Update actor  $\theta_i \leftarrow \theta_i + \alpha_{\pi} \frac{1}{|\beta|} \sum_b \nabla_{\theta_i} J^b(\theta_i)$ 
25:  end for
26: end for
27: Output:  $\{\pi_{\theta_i}\}_{i \in \mathcal{I}}$ 

```

---

to obtain the probability of a response based on the current policy. At dialog turn  $t$ , for LLM agent  $i$ , the probability of an response  $a_{i,t}$  given  $h_{i,t}$  under policy  $\pi_{\theta_i}$  factorizes autoregressively as  $\pi_{\theta_i}(a_{i,t}|h_{i,t}) = \prod_{\mu=1}^M \pi_{\theta_i}(a_{i,t_{\mu}}|h_{i,t}, a_{i,t_{<\mu}})$ , where  $a_{i,t_{\mu}}$  denotes the  $\mu^{\text{th}}$  token in  $a_{i,t}$ , and  $a_{i,t_{<\mu}}$  is the token prefix up to  $\mu$ . TF computes the conditional distributions for all target tokens in parallel under a causal mask, and thus the log-probability of the entire sequence can be efficiently obtained in one pass by summing the log-probabilities of each token. TF passes are applied twice in Alg. 1, when agents generate responses to roll out (line 8), and when they update their policies (line 20).

### 5.3. Algorithm

Fig. 1b and Alg. 1 illustrate the training procedure of CoLLM-CC. For each rollout, we first sample a task from

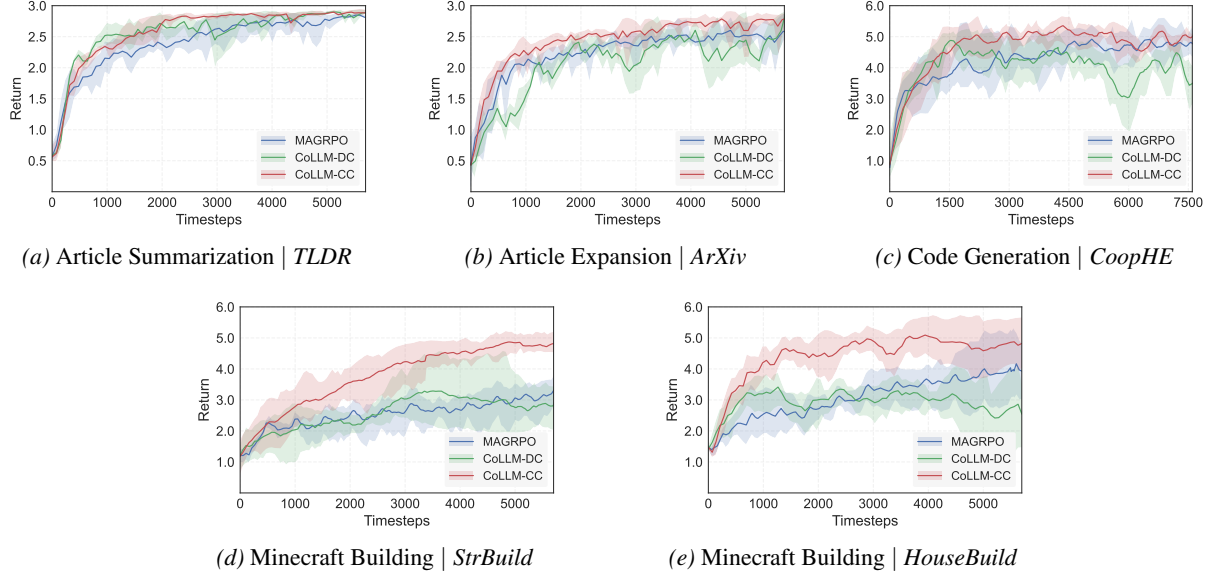


Figure 2. Evaluation results of MAGRPO, CoLLM-DC, and CoLLM-CC across article writing, code generation, and game-playing tasks over 5 runs. The y-axis shows expected return, with limits (min/max) indicating the return scale for each task. Curves are smoothed using a time-weighted exponential moving average. Shaded regions denote 95% bootstrapped confidence intervals.

$\mathcal{D}$ , initialize the agents’ prompts as  $\mathbf{o}_0 \leftarrow \{o_{1,0}, \dots, o_{n,0}\}$ , and obtain the initial global information  $m_0$  from the environment. At each turn  $t \in [0, H)$ , each agent generates a response  $a_{i,t} \sim \pi_{\theta_i}(\cdot|h_{i,t})$  conditioned on its local history  $h_{i,t}$ . A TF pass is applied right after for each agent to compute the probability  $\pi_{\theta_i}(a_{i,t}|h_{i,t})$  of  $a_{i,t}$  under  $\pi_{\theta_i}(\cdot|h_{i,t})$ . All agents’ responses form a solution to the task  $\mathbf{a}_t \leftarrow \{a_{1,t}, \dots, a_{n,t}\}$ , and the joint policy can be constructed by  $\pi_{\theta} \leftarrow \{\pi_{\theta_1}, \dots, \pi_{\theta_n}\}$ . This joint policy is recorded as a behavior policy  $\pi_{\theta_{\text{old}}}(\mathbf{a}_t|\mathbf{h}_t) \leftarrow \pi_{\theta}(\mathbf{a}_t|\mathbf{h}_t)$ .

Executing  $\mathbf{a}_t$  produces a joint reward  $r_t$  from the reward model and triggers the evolution of the environment. The users and systems provide new prompts  $o_{i,t+1}$ , and the global information is updated as  $m_{t+1}$ . Each agent updates its history as  $h_{i,t+1} \leftarrow \{h_{i,t}, a_{i,t}, o_{i,t+1}\}$ . The resulting transition  $(\mathbf{h}_t, m_t, \mathbf{a}_t, r_t, \mathbf{o}_{t+1}, \pi_{\theta_{\text{old}}}(\mathbf{a}_t|\mathbf{h}_t))$ , including prompts, global information, responses, reward, new prompts, new global information, and the behavior policy, is stored into the replay buffer  $\mathcal{B}$  for subsequent training.

At each training epoch, a minibatch  $\beta$  of joint transitions is drawn from the replay buffer  $\mathcal{B}$ . For each sample indexed by  $b$ ,  $(\mathbf{h}_t^b, m_t^b, \mathbf{a}_t^b, r_t^b, \mathbf{o}_{t+1}^b, m_{t+1}^b, \pi_{\theta_{\text{old}}}^b(\mathbf{a}_t^b|\mathbf{h}_t^b)) \in \beta$ , the temporal difference  $\mathcal{L}^b(\phi, m_t^b)$  is calculated according to Equation 5 by the structure shown in Fig 1c. The probability of outputting  $a_{i,t}^b$  by each agent under its current policy  $\pi_{\theta_i}(a_{i,t}^b|h_{i,t}^b)$  is calculated via a TF pass, which is then used to compute policy gradient  $\nabla_{\theta_i} J^b(\theta_i)$  according to Equation 4 in Figure 1a. The gradient of all samples in  $\beta$  is averaged to update the centralized critic  $\phi$  and all actors  $\theta_i$ ,  $\forall i \in \mathcal{I}$ , respectively.

## 6. Experiments

We evaluate MAAC on 3 domains: document processing, programming, and language-based games. Experiment settings, additional results, the prompt and reward design, resources used, our code and dataset are provided in Appendix C, D, E, F, G, H, respectively.

### 6.1. Setup

**Writing Collaboration** Processing long documents is time-consuming, whereas parallel execution among decentralized agents can substantially improve efficiency. We frame the common collaborative writing into 2 representative tasks. In *TLDR* summarization, 2 *Qwen3-1.7B* agents summarize Reddit posts from the `prompt` field in *TLDR*. One acts as a high-level summarizer producing a concise paragraph, and one serves as a detailed summarizer providing more comprehensive information. In *arXiv* expansion, the same agents expand paper abstracts from the `abstract` field of *arXiv-public-datasets* into full introductions, where one outlines the research background and motivation, and the other presents the proposed methods and experiments. The writing quality is evaluated using a weighted sum of 3 metrics with task-specific hyperparameters. The structure assesses the length ratio of generated responses, encouraging agents to contribute fairly to maximize parallel execution. Style consistency is quantified by a Jaccard similarity score to ensure a uniform tone throughout the article. Logical coherence is measured by a logarithmic function of the frequency of transition words, encouraging natural, smooth transitions between paragraphs.

Table 1. Response time (seconds), cost (tokens/agent/turn), and performance (%) of MAAC and other baselines (single-model, multi-agent test-time interaction, and MA-REINFORCE) on article writing, code generation, and language-based games. Adj denotes the same-texture adjacency rate, and HP denotes the average remaining health points. Response time is measured on a *NVIDIA GeForce RTX 5090*. **Bolds** and underlines indicate the best performance across all baselines and MARL baselines on each dataset. Results are averaged over 5 runs.

| Method     | TLDR       |            |             | arXiv      |            |             | CoopHE     |            |             | StrBuild   |            |            |             | HouseBuild  |            |              |             |
|------------|------------|------------|-------------|------------|------------|-------------|------------|------------|-------------|------------|------------|------------|-------------|-------------|------------|--------------|-------------|
|            | Time       | Cost       | Score       | Time       | Cost       | Score       | Time       | Cost       | Pass        | Time       | Cost       | Adj        | IoU         | Time        | Cost       | HP           | IoU         |
| Raw Model  | 5.0        | 465        | 30.3        | 5.1        | 472        | 44.6        | 2.5        | 90         | 56.3        | 10.6       | 427        | 0.9        | 36.6        | 22.6        | 1016       | 99.6         | 43.2        |
| GRPO       | 4.1        | 387        | 91.7        | 4.2        | 398        | 91.0        | 2.5        | <b>88</b>  | 61.8        | 10.3       | 411        | <b>0.4</b> | 46.1        | 22.0        | 890        | <b>100.0</b> | 54.6        |
| AC         | 4.0        | 374        | 94.5        | 4.3        | 392        | <b>95.3</b> | 2.5        | 91         | 62.5        | 10.3       | 413        | <b>0.4</b> | 49.8        | 22.1        | 904        | <b>100.0</b> | <b>55.9</b> |
| Parallel   | 2.3        | 244        | 22.9        | 2.3        | 246        | 49.0        | <u>2.3</u> | 138        | 50.0        | 9.4        | 232        | 15.7       | 5.9         | 19.2        | 502        | 21.8         | 46.1        |
| Pipeline   | 4.3        | 238        | 21.7        | 3.9        | 203        | 57.8        | 2.6        | 177        | 62.5        | 9.8        | 246        | 12.9       | 18.7        | 20.3        | 488        | 30.6         | 41.3        |
| Discussion | 4.6        | 234        | 22.3        | 4.8        | 251        | 54.3        | 2.9        | 191        | 25.0        | 10.3       | 236        | 16.2       | 6.5         | 21.0        | 510        | 27.6         | 38.1        |
| MAGRPO     | <u>1.8</u> | <u>178</u> | 93.5        | 2.0        | 201        | 93.1        | <u>2.3</u> | 132        | 74.3        | 9.4        | 226        | 13.3       | 50.6        | 19.2        | 446        | 80.2         | 50.9        |
| CoLLM-DC   | 1.9        | 194        | <b>95.4</b> | 2.0        | 196        | 94.1        | 2.5        | 161        | 59.1        | <u>9.3</u> | <b>182</b> | 7.6        | 44.6        | 19.4        | 470        | 43.8         | 46.8        |
| CoLLM-CC   | <u>1.8</u> | 181        | 95.2        | <u>1.9</u> | <b>188</b> | <u>95.0</u> | 2.6        | <u>166</u> | <u>75.2</u> | 9.5        | 239        | <u>7.3</u> | <b>68.5</b> | <u>19.0</u> | <b>442</b> | <u>86.4</u>  | <u>52.7</u> |

**Coding Collaboration** Leveraging multiple LLM agents for collaborative coding has emerged as a promising paradigm (Talebirad & Nadiri, 2023; NVIDIA, 2024; Qian et al., 2024). We use a code generation task to demonstrate this concept. To test the collaboration between heterogeneous agents, an auxiliary *Qwen2.5-Coder-3B* assists a primary *Qwen3-4B-Instruct* generator in solving basic programming tasks. The auxiliary agent intends to implement lightweight utilities that support the primary agent to produce the core logic. As many tasks in *HumanEval* and *MBPP* (Chen et al., 2021; Austin et al., 2021) are atomic and cannot meaningfully benefit from cooperation (e.g., `add(x, y)`), optimizing collaborative behaviors on such instances introduces substantial noise. Therefore, we construct *CoopHumanEval* dataset (*CoopHE*), which focuses on problems that naturally admit cooperative decomposition. In *CoopHE*, the auxiliary function is named `aux`, and the main function signature is provided in the `prompt` field of the problem description. Agents interact in a multi-turn environment,  $H = 2$ . Agents’ outputs are concatenated into a joint program. They receive feedback from static analysis (abstract syntax trees) or dynamic execution (sandbox tests), then proceed to the next turn. The episode terminates if the program passes all tests and the main effectively utilizes the auxiliary function. We use the average pass rate as the evaluation metric. Pass@k results are shown in Appendix D.

**Language-based Games** Language-based games comprise diverse, flexible interactive environments, where agents act and coordinate via language instructions (Sarkar et al., 2025; Zhu et al., 2025). We consider 2 tasks from *Minecraft* to evaluate LLM collaboration. In *StrBuild*, a *Qwen2.5-3B-Instruct* and a *Qwen3-4B-Instruct* agents collaboratively construct string-like structures (e.g., “ICML” in Fig. 3a). Each agent is provided with 2 different block types from wood, stone, concrete, and obsidian, each exhibiting distinct properties, e.g., accessibility and defensive

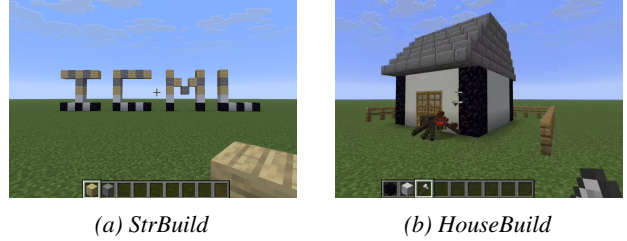


Figure 3. Screenshots of building tasks in Minecraft. (a) *StrBuild*: The LLM agent with wood outputs a `/setblock 12 5 5 minecraft:birch_planks` game instruction to complete the building in “ICML” shape. (b) *HouseBuild*: The LLM agent outputs `/damage @e[type=spider,limit=1] 6 minecraft:player_attack` to attack a mob, while building a cubic concrete house with a wooden door, 4 obsidian pillars, and a triangular-prism stone roof.

capability. Each agent’s resource is limited. The target building is expected to match the exact string while maintaining an even material distribution to enhance structural robustness against potential threats. *HouseBuild* involves a higher level of interactivity and stochasticity. The same agents collaboratively construct a house that conforms to predefined architectural specifications (e.g., Fig. 3b) while defending against attacks from hostile mobs (spiders). Since adversaries can actively interfere with agents, effective coordination requires allocating effort between construction and threat mitigation. *StrBuild* and *HouseBuild* unfold in 4 turns,  $H = 4$ . A simulated player gives instructions for incomplete building after each turn. For *StrBuild*, we adopt the adjacency rate of same-texture blocks as a task-specific metric, where a lower value corresponds to a more evenly distributed texture across the buildings. We use the average health point of players (HP) as an indicator of whether the attacks are successfully repelled in *HouseBuild*. Intersection over Union (IoU) serves as an evaluation metric for both building tasks, with higher values indicating greater compliance with the given specifications.

## 6.2. Baselines

We evaluate the performance of CoLLM-DC and CoLLM-CC against 3 categories of baselines: single-model methods, multi-agent test-time interaction, and other MARL methods.

**Single-Model Methods** Single-model baselines contain the results from a larger base model or its fine-tuned variants. We select a comparable-size model as the sum of MAS: *Qwen3-4B-Instruct* for writing tasks, *Qwen2.5-7B Coder* and *Instruct* models for code generation and Minecraft game-playing, respectively. The reward model is the same as that used in MARL methods, except for metrics specifically designed for cooperation. We set all single-model baselines to be single-turn, as the external feedback for single-model and multi-agent approaches differ significantly.

**Multi-Agent Test-Time Interaction Methods** We compare our method against 3 prompt-based multi-agent interaction baselines, where the same agents operate only through prompts (Wu et al., 2023). In the parallel baseline, agents act independently without explicit communication. The sequential pipeline baseline represents a one-way communication setting where agents act in turn and can later observe the inputs and outputs of earlier agents. The discussion baseline frames the multi-agent debate (Du et al., 2023), where agents engage in two-way, asynchronous communication. Each agent can access the others’ outputs in the next turn and generate its response accordingly. To make a fair comparison, we set the number of message-passing rounds to the same across multi-agent methods and keep the prompt adjustment minimal.

**MARL Methods** We compare CoLLM-DC and CoLLM-CC with a representative MA-REINFORCE approach (Section 4.1), Multi-Agent Group Relative Policy Optimization (MAGRPO) (Liu et al., 2026c). In MAGRPO, the group relative baseline  $b^{\text{GRP}}(\mathbf{h}_t) = \frac{1}{K} \sum_{k=1}^K G_t^k$  is the average return for  $K$  i.i.d. responses  $\{\mathbf{a}^1, \dots, \mathbf{a}^K\}$  on each joint history  $\mathbf{h}_t$ . We set  $K = 4$  for short-horizon writing and coding tasks, and  $K = 2$  for Minecraft tasks due to the rapid increase in the number of required samples (Proposition 4.3). The same learning rate is applied to update each sample across all 3 methods to ensure a fair comparison of sample efficiency (Appendix D). We use the task completion progress and the current turn number as  $m_t$  in both CC and DC for consistency, though such information is often unavailable to DC in standard MARL.

## 6.3. Evaluation Results

The performance comparison between MARL methods and other baselines is presented in Table 1, where the evaluation curves of MAGRPO (blue), CoLLM-DC (green), and CoLLM-CC (red) are presented in Fig. 2.

**Writing Collaboration** In writing tasks, single-model methods achieve strong performance but have low inference speed and high cost due to their larger model size. Since the raw models are not specifically optimized for coordination-centric objectives, prompt-based multi-agent methods produce low-quality solutions. Both RL and MARL fine-tuning guide agents to generate concise, high-quality content, leading to shorter response times, lower token usage, and improved performance. MAGRPO and CoLLM-CC achieve similar performance after convergence, indicating that  $K = 4$  samples are sufficient for accurate value estimation. MAGRPO has slightly higher variance and lower sample efficiency, as shown by a larger shaded region and slower convergence in Fig. 2a and 2b. CoLLM-DC struggles to achieve stable convergence in *arXiv* expansion, as DC based on local information cannot provide stationary signals for gradient estimates.

**Coding Collaboration** In coding collaboration, fine-tuning *Qwen2.5-Coder-7B* yields limited improvement ( $\leq 6.2\%$ ), as coding logics vary substantially across training and test problems and no general strategy consistently improves pass rates across diverse tasks. Parallel execution and one-round discussion perform poorly, as agents lack timely information to accurately assess the correctness and functionality of others’ functions. Sequential generation achieves performance comparable to the single-model baseline but incurs substantially higher inference latency because agents must wait for others’ responses before proceeding. With  $K = 4$ , MAGRPO achieves comparably high pass rates 74.3% as CoLLM-CC 75.2%. The main generator provides fallback solutions for potential vulnerabilities in the auxiliary, and it effectively uses external feedback to correct errors. However, MAGRPO requires significantly more samples to converge in Fig. 2c, reaching stability at approximately 5000 timesteps, compared to 2000 timesteps for CoLLM-CC. CoLLM-DC exhibits oscillations in later stages (4500 timesteps) and thus cannot perform well, with only 59.1% pass rate. This instability is caused by sparse rewards in code generation, where a single incorrect token can invalidate functionality and induce abrupt reward drops.

**Minecraft Games** In Minecraft games, the fine-tuned single model achieves the lowest same-texture adjacency rate 0.4-0.9% in *StrBuild* and the full health points in *HouseBuild*. This is because the agents do not need to infer others’ material choices or attack behavior, thus the task is relatively easier. Prompt-based multi-agent methods perform particularly poorly in these domains, likely because the raw models *Qwen2.5-3B-Instruct* and *Qwen3-4B-Instruct* are not specifically optimized with Minecraft building instructions. CoLLM-CC achieves the best IoU 68.5% in *StrBuild* by leveraging external hints and outperforms all methods except the single model fine-tuned by AC in *HouseBuild* with

Table 2. Performance of MARL algorithms for decentralized LLM collaboration on Minecraft tasks with  $n = 2$  and  $n = 3$  agents. The  $n = 2$  setting, marked with †, serves as the control group. For  $n = 3$ , subscripts report the numerical change relative to the corresponding  $n = 2$  result; arrows indicate whether the metric value increases or decreases, while colors indicate whether the change improves (red) or degrades (blue) performance. Results are averaged over 5 runs. The best results among the three methods are marked in **bold**.

| Method   | StrBuild ( $n = 2$ ) |              | StrBuild ( $n = 3$ )      |                            | HouseBuild ( $n = 2$ ) |              | HouseBuild ( $n = 3$ )     |                            |
|----------|----------------------|--------------|---------------------------|----------------------------|------------------------|--------------|----------------------------|----------------------------|
|          | Adj                  | IoU          | Adj                       | IoU                        | HP                     | IoU          | HP                         | IoU                        |
| MAGRPO   | 13.3†                | 50.6†        | 8.7 <sub>↓4.6</sub>       | 38.1 <sub>↓12.5</sub>      | 80.2†                  | 50.9†        | 84.0 <sub>↑3.8</sub>       | 46.3 <sub>↓4.6</sub>       |
| CoLLM-DC | 7.6†                 | 44.6†        | 9.4 <sub>↑1.8</sub>       | 34.5 <sub>↓10.1</sub>      | 43.8†                  | 46.8†        | 50.6 <sub>↑6.8</sub>       | 43.7 <sub>↓3.1</sub>       |
| CoLLM-CC | <b>7.3†</b>          | <b>68.5†</b> | <b>6.9<sub>↓0.4</sub></b> | <b>75.4<sub>↑6.9</sub></b> | <b>86.4†</b>           | <b>52.7†</b> | <b>94.2<sub>↑7.8</sub></b> | <b>61.3<sub>↑8.6</sub></b> |

IoU 52.7%. In Fig 2d and 2e, MAGRPO and CoLLM-DC perform substantially worse than CoLLM-CC. Due to the rapid growth of the rollout tree, a smaller sampling  $K = 2$  is needed, resulting in slower convergence and higher-variance updates, as discussed in Appendix D. CoLLM-DC fails to converge on these tasks since non-stationarity accumulates across 4 turns, leading to more unstable value estimates.

**Results in Larger-Scale MAS** We also evaluate our methods in a larger-scale setting with  $n = 3$  agents. We keep the reward design unchanged and introduce an additional agent with the same prompt structure. The results are shown in Table 2. Compared with the  $n = 2$  setting, all methods achieve higher HP on *HouseBuild*, suggesting that additional agents can provide better defense against spiders. On *StrBuild*, MAGRPO achieves lower Adj  $\downarrow 4.6\%$ , but this comes at the cost of less accurate adherence to the building specification, i.e., lower IoU  $\downarrow 12.5\%$ . Moreover, MAGRPO incurs substantially longer training time (Appendix D), as suggested in Proposition 4.3. CoLLM-DC also shows worse performance when scaling to more agents, with  $\uparrow 1.8\%$  Adj and  $\downarrow 10.1\%$  on *StrBuild* and  $\downarrow 3.1\%$  on *HouseBuild*. This is because each decentralized critic observes only a limited portion of the global state, making the non-stationarity issue more severe. In contrast, CoLLM-CC achieves the strongest overall performance when  $n = 3$ , with both lower Adj  $\downarrow 0.4\%$ , higher HP  $\uparrow 7.8\%$ , and IoU  $\uparrow 6.9\%$  and  $\uparrow 8.6\%$  on *StrBuild* and on *HouseBuild*, respectively. These results indicate that CoLLM-CC can effectively support cooperative learning in larger-scale MAS under decentralized execution.

## 7. Conclusion

Decentralized LLM collaboration accelerates inference and enables flexible deployment, making it promising. We developed centralized critic (CoLLM-CC) and decentralized critic (CoLLM-DC) methods to optimize cooperation among decentralized LLMs. Evaluations on writing, coding, and game-playing tasks show that MARL-based methods can achieve equal or better performance than a single larger model. Among MARL methods, MA-REINFORCE and CoLLM-DC struggle in long-horizon, sparse-reward settings due to low sample efficiency and convergence issues,

while CoLLM-CC achieves the best performance with the lowest variance and highest sample efficiency.

**Limitations** Our work has several limitations. First, although our MAAC methods (CoLLM-DC and CoLLM-CC) can in principle be applied to online learning, we do not extend our experiments to longer-horizon tasks. This is because MAGRPO has low sample efficiency, making fair comparisons prohibitively expensive in such settings. Extending these methods to more complex agentic domains remains an important direction for future work. Second, as discussed in Appendix B, more efficient variants of CoLLM-DC are possible. However, these variants may introduce additional challenges, such as gradient conflicts. It remains worth exploring whether these issues can be mitigated, and whether similar efficiency improvements can be incorporated into centralized-critic methods. Finally, our methods focus on a strictly decentralized setting with concurrent execution and no inter-agent communication. Designing methods under relaxed communication or coordination constraints remains an open question.

## Acknowledgment

This work was partially funded by NSF grants #2409351 and #2525087. It used Delta and DeltaAI computing resources at the National Center for Supercomputing Applications through allocation CIS251326 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support program, which is supported by NSF grants #2138259, #2138286, #2138307, #2137603, and #2138296.

This work was completed in part using the Explorer Cluster, supported by Northeastern University’s Research Computing team. Shuo Liu received computing support from Lambda’s Research Grant Program.

## Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

## References

- Achiam, J., Adler, S., Agarwal, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Ahmadian, A., Cremer, C., Gallé, M., Fadaee, M., Kreutzer, J., Pietquin, O., Üstün, A., and Hooker, S. Back to basics: Revisiting reinforce style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.
- Albrecht, S. V., Christianos, F., and Schäfer, L. *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. URL <https://www.marl-book.com>.
- Amato, C. An introduction to centralized training for decentralized execution in cooperative multi-agent reinforcement learning. *arXiv preprint arXiv:2409.03052*, 2024.
- Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- Baisero, A., Bhati, R., Liu, S., Pillai, A., and Amato, C. Fixing incomplete value function decomposition for multi-agent reinforcement learning. *arXiv preprint arXiv:2505.10484*, 2025.
- Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., et al. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*, 2025.
- Chen, M., Tworek, J., Jun, H., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, Y., Arkin, J., Zhang, Y., Roy, N., and Fan, C. Scalable multi-robot collaboration with large language models: Centralized or decentralized systems? In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4311–4317. IEEE, 2024.
- Chen, Y., Yan, L., Sun, W., Ma, X., Zhang, Y., Wang, S., Yin, D., Yang, Y., and Mao, J. Improving retrieval-augmented generation through multi-agent reinforcement learning. *arXiv preprint arXiv:2501.15228*, 2025.
- Chen, Y., Yan, L., Yang, Z., Zhang, E., Zhao, J., Wang, S., Yin, D., and Mao, J. Beyond monolithic architectures: A multi-agent search and knowledge optimization framework for agentic search. *arXiv preprint arXiv:2601.04703*, 2026.
- De Witt, C. S., Gupta, T., Makoviichuk, D., Makovychuk, V., Torr, P. H., Sun, M., and Whiteson, S. Is independent learning all you need in the starcraft multi-agent challenge? *arXiv preprint arXiv:2011.09533*, 2020.
- Ding, Z. and Ye, W. Treegrpo: Tree-advantage grpo for online rl post-training of diffusion models. *arXiv preprint arXiv:2512.08153*, 2025.
- Douillard, A., Feng, Q., Rusu, A. A., Chhaparia, R., Donchev, Y., Kuncoro, A., Ranzato, M., Szlam, A., and Shen, J. Diloco: Distributed low-communication training of language models. *arXiv preprint arXiv:2311.08105*, 2023.
- Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.
- Estornell, A. and Liu, Y. Multi-LLM debate: Framework, principals, and interventions. In *NeurIPS*, 2024.
- Feng, L., Zheng, L., He, S., Zhang, F., and An, B. Dr. mas: Stable reinforcement learning for multi-agent llm systems. *arXiv preprint arXiv:2602.08847*, 2026.
- Feng, S., Wang, Z., Goyal, P., Wang, Y., Shi, W., Xia, H., Palangi, H., Zettlemoyer, L., Tsvetkov, Y., Lee, C.-Y., et al. Heterogeneous swarms: Jointly optimizing model roles and weights for multi-LLM systems. *arXiv preprint arXiv:2502.04510*, 2025.
- Foerster, J., Farquhar, G., Afouras, T., Nardelli, N., and Whiteson, S. Counterfactual multi-agent policy gradients. In *Proceedings of the AAAI Conference on Artificial Intelligence*, April 2018.
- Gao, Y., Song, Z., and Yin, J. Gradientcoin: A peer-to-peer decentralized large language models. *arXiv preprint arXiv:2308.10502*, 2023.
- Ghosh, S. *Distributed systems: an algorithmic approach*. Chapman and Hall/CRC, 2006.
- Guo, D., Yang, D., Zhang, H., et al. Deepseek-r1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.
- Hao, H., Han, M., Li, C., Li, Z., and Chang, X. Conav: Collaborative cross-modal reasoning for embodied navigation. *arXiv preprint arXiv:2505.16663*, 2025.

- Hong, H., Yin, J., Wang, Y., Liu, J., Chen, Z., Yu, A., Li, J., Ye, Z., Xiao, H., Chen, Y., et al. Multi-agent deep research: Training multi-agent systems with m-grpo. *arXiv preprint arXiv:2511.13288*, 2025.
- Hu, J., Liu, J. K., Xu, H., and Shen, W. Reinforce++: Stabilizing critic-free policy optimization with global advantage normalization, 2025.
- Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M., Lee, H., Ngiam, J., Le, Q. V., Wu, Y., et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- Ji, Y., Ma, Z., Wang, Y., Chen, G., Chu, X., and Wu, L. Tree search for llm agent reinforcement learning. *arXiv preprint arXiv:2509.21240*, 2025.
- Konda, V. and Tsitsiklis, J. Actor-critic algorithms. In Solla, S., Leen, T., and Müller, K. (eds.), *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999.
- Lee, K. M., Ganapathi Subramanian, S., and Crowley, M. Investigation of independent reinforcement learning algorithms in multi-agent environments. *Frontiers in Artificial Intelligence*, 5:805823, 2022.
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- Li, Y., Gu, Q., Wen, Z., Li, Z., Xing, T., Guo, S., Zheng, T., Zhou, X., Qu, X., Zhou, W., et al. Treepo: Bridging the gap of policy optimization and efficacy and inference efficiency with heuristic tree-based modeling. *arXiv preprint arXiv:2508.17445*, 2025.
- Li, Z., Xu, T., Zhang, Y., Yu, Y., Sun, R., and Luo, Z.-Q. Remax: A simple, effective, and efficient method for aligning large language models. *arXiv preprint arXiv:2310.10505*, 2023.
- Liao, J., Wen, M., Wang, J., and Zhang, W. Marft: Multi-agent reinforcement fine-tuning. *arXiv preprint arXiv:2504.16129*, 2025.
- Lifshitz, S., McIlraith, S. A., and Du, Y. Multi-agent verification: Scaling test-time compute with multiple verifiers. *arXiv preprint arXiv:2502.20379*, 2025.
- Liu, B., Guertler, L., Yu, S., Liu, Z., Qi, P., Balcells, D., Liu, M., Tan, C., Shi, W., Lin, M., et al. Spiral: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning. *arXiv preprint arXiv:2506.24119*, 2025a.
- Liu, M., Jiang, L., Liang, Y., Du, S. S., Choi, Y., Althoff, T., and Jaques, N. Chasing moving targets with online self-play reinforcement learning for safer language models. *arXiv preprint arXiv:2506.07468*, 2025b.
- Liu, R., Shen, Y., Gao, P., Tokekar, P., and Lin, M. C. Caml: Collaborative auxiliary modality learning for multi-agent systems. *Advances in Neural Information Processing Systems*, 38:144064–144087, 2026a.
- Liu, S., Li, X., and Amato, C. Cross-modal navigation with multi-agent reinforcement learning. *arXiv preprint arXiv:2605.06595*, 2026b.
- Liu, S., Liang, Z., Lyu, X., and Amato, C. LLM collaboration with multi-agent reinforcement learning. In *Proceedings of the 40th Annual AAAI Conference on Artificial Intelligence*, 2026c.
- Liu, S.-Y., Dong, X., Lu, X., Diao, S., Belcak, P., Liu, M., Chen, M.-H., Yin, H., Wang, Y.-C. F., Cheng, K.-T., et al. Gdpo: Group reward-decoupled normalization policy optimization for multi-reward rl optimization. *arXiv preprint arXiv:2601.05242*, 2026d.
- Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- Lowe, R., Wu, Y. I., Tamar, A., Harb, J., Abbeel, P., and Mor-datch, I. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in neural information processing systems*, 30, 2017.
- Lyu, X., Xiao, Y., Daley, B., and Amato, C. Contrasting centralized and decentralized critics in multi-agent reinforcement learning. *arXiv preprint arXiv:2102.04402*, 2021.
- Lyu, X., Baisero, A., Xiao, Y., Daley, B., and Amato, C. On centralized critics in multi-agent reinforcement learning. *Journal of Artificial Intelligence Research*, 77:295–354, 2023.
- NVIDIA. Introduction to llm agents, 2024. URL <https://developer.nvidia.com/blog/introduction-to-llm-agents/>.
- Oliehoek, F. A. and Amato, C. *A Concise Introduction to Decentralized POMDPs*. Springer, 2016. doi: 10.1007/978-3-319-28929-8.
- Peshkin, L., Kim, K.-E., Meuleau, N., and Kaelbling, L. P. Learning to cooperate via policy search. *arXiv preprint cs/0105032*, 2001.

- Qi, M., Zhu, T., Zhang, L., Li, N., and Zhou, W. Towards transparent and incentive-compatible collaboration in decentralized llm multi-agent systems: A blockchain-driven approach. *arXiv preprint arXiv:2509.16736*, 2025.
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15174–15186, 2024.
- Rashid, T., Samvelyan, M., de Witt, C. S., Farquhar, G., Foerster, J., and Whiteson, S. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178):1–51, 2020.
- Sarkar, B., Xia, W., Liu, C. K., and Sadigh, D. Training language models for social deduction with multi-agent reinforcement learning. *arXiv preprint arXiv:2502.06060*, 2025.
- Shao, Z., Wang, P., Zhu, Q., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shoham, Y. and Leyton-Brown, K. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, Cambridge, UK, 2009.
- Skreta, M., Yoshikawa, N., Arellano-Rubach, S., Ji, Z., Kristensen, L. B., Darvish, K., Aspuru-Guzik, A., Shkurti, F., and Garg, A. Errors are useful prompts: Instruction guided task programming with verifier-assisted iterative prompting. *arXiv preprint arXiv:2303.14100*, 2023.
- Stone, P. and Veloso, M. Multiagent systems: A survey from a machine learning perspective. *Autonomous Robots*, 8(3):345–383, July 2000.
- Subramaniam, V., Du, Y., Tenenbaum, J. B., Torralba, A., Li, S., and Mordatch, I. Multiagent finetuning: Self improvement with diverse reasoning chains. *arXiv preprint arXiv:2501.05707*, 2025.
- Sunehag, P., Lever, G., Gruslys, A., Czarnecki, W. M., Zambaldi, V., Jaderberg, M., Lanctot, M., Sonnerat, N., Leibo, J. Z., Tuyls, K., et al. Value-decomposition networks for cooperative multi-agent learning. *arXiv preprint arXiv:1706.05296*, 2017.
- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, 1998.
- Talebirad, Y. and Nadiri, A. Multi-agent collaboration: Harnessing the power of intelligent llm agents. *arXiv preprint arXiv:2306.03314*, 2023.
- Uesato, J., Kushman, N., Kumar, R., Song, F., Siegel, N., Wang, L., Creswell, A., Irving, G., and Higgins, I. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.
- Van der Hoek, W. and Wooldridge, M. Multi-agent systems. *Foundations of Artificial Intelligence*, 3:887–928, 2008.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Waldo, J., Wyant, G., Wollrath, A., and Kendall, S. A note on distributed computing. In *International Workshop on Mobile Object Systems*, pp. 49–64. Springer, 1996.
- Wang, J., Ren, Z., Liu, T., Yu, Y., and Zhang, C. Qplex: Duplex dueling multi-agent q-learning. In *Proceedings of the International Conference on Learning Representations*, 2021.
- Wang, S., Li, P., Fu, Y., Liu, K., Li, F., Liu, Y., Sun, X., Li, Z., Zhao, S., Zhao, J., et al. Marti-mars<sup>2</sup>: Scaling multi-agent self-search via reinforcement learning for code generation. *arXiv preprint arXiv:2602.07848*, 2026a.
- Wang, T., Li, X., Lu, F., Gong, T., Dong, J., Xue, W., Qu, S., Bai, C., and Chen, G. Conavbench: Collaborative long-horizon vision-language navigation benchmark. In *The Fourteenth International Conference on Learning Representations*, 2026b.
- Weiss, G. *Multiagent systems: a modern approach to distributed artificial intelligence*. MIT press, 1999.
- Wu, L., Liu, X., Shi, T., Ye, Z., and Song, D. Deserve: Towards affordable offline llm inference via decentralization. *arXiv preprint arXiv:2501.14784*, 2025a.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Wu, S., Galley, M., Peng, B., Cheng, H., Li, G., Dou, Y., Cai, W., Zou, J., Leskovec, J., and Gao, J. Collabllm: From passive responders to active collaborators. *arXiv preprint arXiv:2502.00640*, 2025b.
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

- Yang, Y., Chai, H., Shao, S., Song, Y., Qi, S., Rui, R., and Zhang, W. Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems. *arXiv preprint arXiv:2504.00587*, 2025a.
- Yang, Z., Guo, Z., Huang, Y., Liang, X., Wang, Y., and Tang, J. Treerpo: Tree relative policy optimization. *arXiv preprint arXiv:2506.05183*, 2025b.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- Yu, C., Velu, A., Vinitisky, E., Gao, J., Wang, Y., Bayen, A., and Wu, Y. The surprising effectiveness of ppo in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*, volume 35, pp. 24611–24624. Curran Associates, Inc., 2022.
- Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Song, S., and Huang, G. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.
- Zhang, K., Liu, R., Zhu, X., Tian, K., Zeng, S., Jia, G., Fan, Y., Lv, X., Zuo, Y., Jiang, C., Liu, Z., Wang, J., Wang, Y., Zhao, R., Hua, E., Wang, Y., Wang, S., Gao, J., Long, X., Sun, Y., Ma, Z., Cui, G., Bai, L., Ding, N., Qi, B., and Zhou, B. MARTI: a framework for multi-agent LLM systems reinforced training and inference, 2025a.
- Zhang, S., Yin, M., Zhang, J., Liu, J., Han, Z., Zhang, J., Li, B., Wang, C., Wang, H., Chen, Y., et al. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. *arXiv preprint arXiv:2505.00212*, 2025b.
- Zhao, W., Yuksekgonul, M., Wu, S., and Zou, J. Sirius: Self-improving multi-agent systems via bootstrapped reasoning. *arXiv preprint arXiv:2502.04780*, 2025a.
- Zhao, Y., Hu, L., Wang, Y., Hou, M., Zhang, H., Ding, K., and Zhao, J. Stronger-mas: Multi-agent reinforcement learning for collaborative llms. *arXiv preprint arXiv:2510.11062*, 2025b.
- Zhu, K., Du, H., Hong, Z., Yang, X., Guo, S., Wang, D. Z., Wang, Z., Qian, C., Tang, R., Ji, H., et al. Multiagent-bench: Evaluating the collaboration and competition of LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8580–8622, 2025.

## A. Proofs

Here we provide the full proofs for the propositions in our paper.

### A.1. Proof of Proposition 4.1

**Proposition.** For each history  $\mathbf{h}_t$  at  $t$ ,  $t \in [0, H)$ , suppose agents sample  $K \geq 1$  i.i.d. joint actions  $\{\mathbf{a}_t^1, \dots, \mathbf{a}_t^K\}$  from  $\pi(\cdot | \mathbf{h}_t)$ . For each  $\mathbf{a}_t^k$ , an independent  $K$ -ary rollout tree is produced to estimate the corresponding expected Monte Carlo return  $G_t^k$ . For agent  $i$ , define  $\bar{g}_{i,t}$  as the averaged gradient estimate over  $\{a_{i,t}^1, \dots, a_{i,t}^K\}$ ,

$$\bar{g}_{i,t} = \frac{1}{K} \sum_{k=1}^K \rho_{i,t}^k \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t}^k | h_{i,t}) G_t^k.$$

Then  $\bar{g}_{i,t}$  is unbiased for the true gradient term  $g_{i,t}^*$ ,

$$\mathbb{E}_{\pi}[\bar{g}_{i,t} | h_{i,t}] = \mathbb{E}_{\pi}[g_{i,t}^* | h_{i,t}].$$

*Proof.* We show that the expectation of gradient estimates  $\bar{g}_{i,t}$  of  $K$ -sampling MA-REINFORCE ( $K \geq 2$ ) is equal to  $g_{i,t}^1$  of MA-REINFORCE with  $K = 1$ .

For agent  $i$  at time  $t$ , define

$$g_{i,t}^k = \rho_{i,t}^k \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t}^k | h_{i,t}) G_t^k.$$

By linearity of expectation,

$$\begin{aligned} \mathbb{E}_{\pi}[G_t^k | h_{i,t}^k] &= \mathbb{E}_{\pi}\left[\frac{1}{K} \sum_{\ell=1}^K G_{t+1}^{\ell} | h_{i,t}^k\right] \\ &= \frac{1}{K} \sum_{\ell=1}^K \mathbb{E}_{\pi}[G_{t+1}^{\ell} | h_{i,t}^k], \end{aligned}$$

where  $\ell$  denotes the sampling index at  $t+1$ . Since at the terminal step  $t = H-1$ ,  $G_{H-1}^k = r_{H-1}^k$ , and  $g_{i,H-1}^k$  is unbiased. We can derive backward that  $g_{i,t}^k$  is unbiased for all  $t$  and  $k$ . Thus, according to Theorem 1 of (Peshkin et al., 2001),  $\bar{g}_{i,t} = g_{i,t}^1$  is unbiased.  $\square$

### A.2. Proof of Proposition 4.2

**Proposition.** Under the setting of Proposition 4.1, for any history  $h_{i,t}$ , we assume the gradient estimates for  $\{a_{i,t}^1, \dots, a_{i,t}^K\}$  are independent and episodes are not terminated early. Let  $\sigma^2$  be the gradient estimate variance when  $K = 1$ , the variance of  $\bar{g}_{i,t}$  satisfies,

$$\text{Var}_{\pi}(\bar{g}_{i,t} | h_{i,t}) = \frac{\sigma^2}{K^{H-t}}.$$

*Proof.* We show that, when the episodes are not terminated early and expand into a full  $K$ -ary tree, and assuming gradient estimates are independent for  $\{a_{i,t}^1, \dots, a_{i,t}^K\}$ , the variance of  $g_{i,t}^K$  scales inversely with  $K$ .

For agent  $i$  at time  $t$ , define

$$g_{i,t}^k = \rho_{i,t}^k \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t}^k | h_{i,t}) G_t^k.$$

Then, by variance decomposition, we have

$$\begin{aligned} \text{Var}(\bar{g}_{i,t} | h_{i,t}) &= \text{Var}\left(\frac{1}{K} \sum_{k=1}^K g_{i,t}^k | h_{i,t}\right) \\ &= \frac{1}{K^2} \sum_{k=1}^K \text{Var}(g_{i,t}^k | h_{i,t}) + \frac{2}{K^2} \sum_{1 \leq k < l \leq K} \text{Cov}(g_{i,t}^k, g_{i,t}^l | h_{i,t}) \end{aligned}$$

where  $k$  and  $l$  are different i.i.d. samples at  $t$ . Since  $\{a_{i,t}^1, \dots, a_{i,t}^K\}$  are i.i.d., we have  $\text{Cov}(g_{i,t}^k, g_{i,t}^l | h_{i,t}) = 0$ . Assuming the episode is not terminated early, at  $t+1$ , we have  $\ell$  new rollouts for each  $a_{i,t}^k$ , and so on, we have  $\text{leaf} = K^{H-1-t}$  leaf nodes and returns at  $t = H-1$ .

$$\begin{aligned} \text{Var}(\bar{g}_{i,t} | h_{i,t}) &= \frac{1}{K} \text{Var}(g_{i,t}^k | h_{i,t}) \\ &= \frac{1}{K} \text{Var}\left(\rho_{i,t}^k \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t}^k | h_{i,t}) G_t^k | h_{i,t}^k\right) \\ &= \frac{1}{K} \text{Var}\left(\rho_{i,t}^k \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t}^k | h_{i,t}) \frac{1}{K} \sum_{\ell=1}^K G_{t+1}^{\ell} | h_{i,t}^k\right) \\ &= \frac{1}{K} \text{Var}\left(\rho_{i,t}^k \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t}^k | h_{i,t}) \frac{\sum_{\text{leaf}=1}^{K^{H-1-t}} G_{H-1}^{\text{leaf}}}{K^{H-1-t}} | h_{i,t}^k\right) \\ &= \frac{1}{K} \frac{1}{(K^{H-1-t})^2} \sum_{\text{leaf}=1}^{K^{H-1-t}} \text{Var}\left(\rho_{i,t}^k \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t}^k | h_{i,t}) G_t^{\text{leaf}} | h_{i,t}^k\right) \\ &\quad + \frac{2}{K} \frac{2}{(K^{H-1-t})^2} \sum_{1 \leq \text{leaf}.1 \leq \text{leaf}.2 \leq K} \text{Cov}(g_{i,t}^{\text{leaf}.1}, g_{i,t}^{\text{leaf}.2} | h_{i,t}^k) \end{aligned}$$

Assuming the  $\{g_{i,t}^1, \dots, g_{i,t}^K\}$  and are independent,<sup>1</sup>  $\text{Cov}(g_{i,t}^{\text{leaf}.1}, g_{i,t}^{\text{leaf}.2} | h_{i,t}^k) = 0$ .

Thus, we have,

$$\begin{aligned} \text{Var}(\bar{g}_{i,t} | h_{i,t}) &= \frac{1}{K^{H-t}} \text{Var}(\rho_{i,t}^k \nabla_{\theta_i} \log \pi_{\theta_i}(a_{i,t}^k | h_{i,t}) G_t^{\text{leaf}} | h_{i,t}^k) \\ &= \frac{\sigma^2}{K^{H-t}} \end{aligned}$$

$\square$

<sup>1</sup>However, this independent gradient assumption is usually hard to satisfy in Dec-POMDP, because dialog histories with the same prefixes usually have positively correlated values.

## B. CoLLM-DC

### Algorithm 2: CoLLM-DC

```

1: Input: Taskset  $\mathcal{D}$ , LLM agents  $\{\pi_{\theta_i}\}_{i \in \mathcal{I}}$ , decentralized
   LLM critics  $\{V_{\phi_i}\}_{i \in \mathcal{I}}$ , learning rates  $\alpha_\pi, \alpha_V$ , discount
    $\gamma$ , horizon  $H$ , replay buffer  $\mathcal{B}$ , training epochs  $E$ 
2: for each episode do
3:   Roll out same as CoLLM-CC in Alg. 1 (line 3-15)
4:   for training epoch  $e = 1, \dots, E$  do
5:     Sample a minibatch of joint transitions  $\beta$  from  $\mathcal{B}$ 
6:     for each agent  $i \in \mathcal{I}$  do
7:       for each sample agent  $i$ 's transition  $\beta_i$ ,
          $(h_{i,t}^b, m_t^b, a_{i,t}^b, r_t^b, o_{i,t+1}^b, m_{t+1}^b \pi_{\theta_{i,\text{old}}}^b(a_{i,t}^b | h_{i,t}^b)) \in \beta_i$  do
8:         Calculate TD loss  $\mathcal{L}_i^b(\phi_i)$  (Eq. 3)
9:         Calculate  $\pi_{\theta_i}(a_{i,t}^b | h_{i,t}^b)$  with TF
10:        Calculate  $\nabla_{\theta_i} J^b(\theta_i)$  (Eq. 2)
11:      end for
12:      Update critic  $\phi_i \leftarrow \phi_i - \alpha_V \frac{1}{|\beta|} \sum_b \nabla_{\phi_i} \mathcal{L}_i^b(\phi_i)$ 
13:      Update actor  $\theta_i \leftarrow \theta_i + \alpha_\pi \frac{1}{|\beta|} \sum_b \nabla_{\theta_i} J^b(\theta_i)$ 
14:    end for
15:  end for
16: end for
17: Output:  $\{\pi_{\theta_i}\}_{i \in \mathcal{I}}$ 

```

Fig. 4 and Alg. 2 show the training procedure of CoLLM-DC. The rollout and actor updates are the same as those in CoLLM-CC (Alg. 1). The only difference is in the training phase. In each training epoch, a minibatch  $\beta_i$  of joint transitions is drawn from  $\mathcal{B}$ . For each sample,  $(h_{i,t}^b, m_t^b, a_{i,t}^b, r_t^b, o_{i,t+1}^b, m_{t+1}^b \pi_{\theta_{i,\text{old}}}^b(a_{i,t}^b | h_{i,t}^b)) \in \beta_i$ , each agent  $i$  computes its own TD loss  $\mathcal{L}_i^b(\phi_i)$  using a decentralized critic conditioned only on its local history, following the structure shown in Fig. 4c. The probability  $\pi_{\theta_i}(a_{i,t}^b | h_{i,t}^b)$  under the current policy is computed by a TF pass and is used to calculate the policy gradient  $\nabla_{\theta_i} J^b(\theta_i)$  according to Eq. 2 in Fig. 4a. The gradients of all samples in  $\beta_i$  are averaged to update each agent's critic  $\phi_i$  and actor  $\theta_i$  independently,  $\forall i \in \mathcal{I}$ .

It is noteworthy that the CoLLM-DC architecture in Fig. 4 has an efficient variant based on parameter sharing. Unlike CoLLM-CC, the agents in Fig. 4a and Fig. 4c learn the same latent representation  $z(h_{i,t})$ , and thus can potentially share a single model to accelerate learning. However, because the optimization objectives in Eq. 3 and 2 are different, such parameter sharing may introduce gradient interference during training.

## C. Experimental Settings

We introduce the experimental settings in Fig. 2 and Table 1.

### C.1. Datasets

We list the preprocessed datasets we used for our training.

- **Writing Collaboration**
  - Training set:
    - \* *TLDR*[0:1000]
    - \* *arXiv*[0:1000]
  - Test set:
    - \* *TLDR*[1000:1100]
    - \* *arXiv*[1000:1100]
- **Code Collaboration**
  - Training set: *CoopHE*[0:66]
  - Test set: *CoopHE*[66:82]
- **Minecraft Game-Playing**
  - Training set:
    - \* *StrBuild*[0:8]
    - \* *HouseBuild*[0:8]
  - Test set:
    - \* *StrBuild*[8:10]
    - \* *HouseBuild*[8:10]

### C.2. Architectures

We list the architectures of the agent and critic models used in our training.

- **Writing Collaboration**
  - Agents
    - \* *Qwen3-1.7B*
    - \* *Qwen3-1.7B*
  - Critic (if applicable): *Qwen3-1.7B*
  - Temperature: 0.7
  - Top- $p$ : 0.9
  - Top- $k$ : null
  - Max output tokens: 256
- **Code Collaboration**
  - Agents
    - \* *Qwen2.5-Coder-3B*
    - \* *Qwen3-4B-Instruct-2507*
  - Critic (if applicable): *Qwen2.5-Coder-3B*
  - Temperature: 0.6
  - Top- $p$ : 0.6
  - Top- $k$ : null
  - Max output tokens: 256
- **Minecraft Game-Playing**

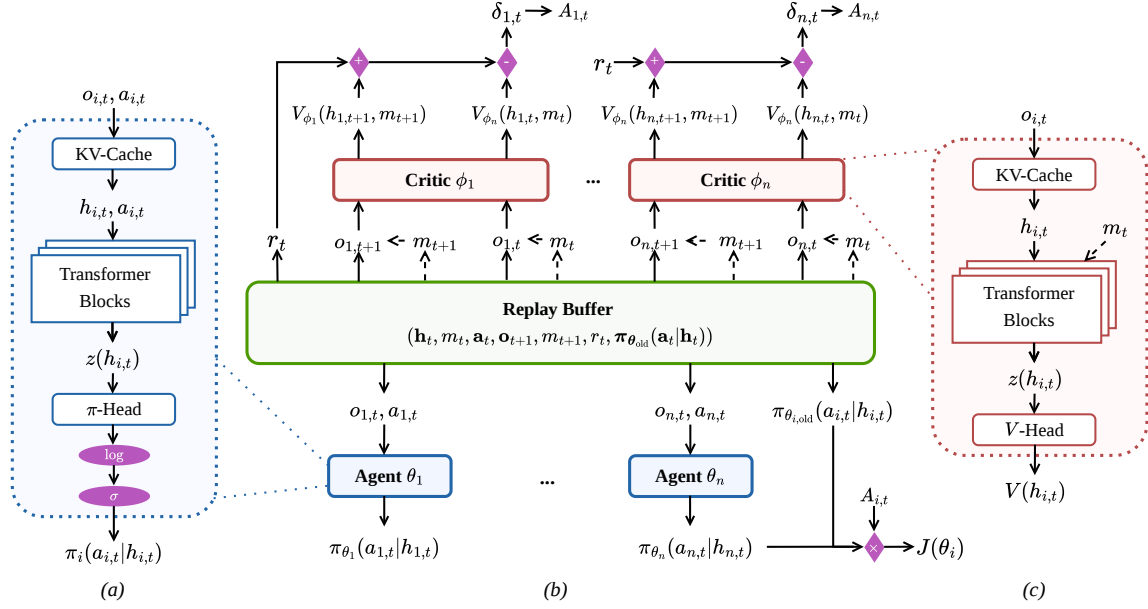


Figure 4. CoLLM-DC framework: (a) The agent structure; (b) The overall decentralized-critic architecture; (c) The critic structure.

- Agents
  - \* *Qwen2.5-3B-Instruct*
  - \* *Qwen3-4B-Instruct-2507*
- Critic (if applicable): *Qwen3-4B-Instruct-2507*
- Temperature: 0.6
- Top- $p$ : 0.6
- Top- $k$ : null
- Max output tokens
  - \* *StrBuild*: 256
  - \* *HouseBuild*: 512

### C.3. Hyperparameters

We show the key hyperparameters used in MAGRPO, CoLLM-DC, and CoLLM-CC.

#### • Writing Collaboration

- Number of turns: 1
- Number of generations: 4
- Rollout buffer size: 4
- Number of train epochs
  - \* MAGRPO: 2
  - \* CoLLM-DC/CoLLM-CC: 20
- Agent learning rate:  $5 \times 10^{-6}$
- Critic learning rate (if applicable):  $3 \times 10^{-6}$
- Advantage clip: 0.2
- Number of evaluation samples: 4

#### • Code Collaboration

- Number of turns: 2
- Number of generations: 4
- Rollout buffer size
  - \* MAGRPO: 16
  - \* CoLLM-DC/CoLLM-CC: 4
- Number of train epochs
  - \* MAGRPO: 8
  - \* CoLLM-DC/CoLLM-CC: 80
- Agent learning rate
  - \* MAGRPO:  $2 \times 10^{-5}$
  - \* CoLLM-DC/CoLLM-CC:  $5 \times 10^{-6}$
- Critic learning rate (if applicable):  $3 \times 10^{-6}$
- Advantage clip: 0.2
- Number of evaluation samples: 4

#### • Minecraft Game-Playing

- Number of turns: 4
- Number of generations: 2
- Rollout buffer size
  - \* MAGRPO: 2
  - \* CoLLM-DC/CoLLM-CC: 1
- Number of train epochs
  - \* MAGRPO: 16
  - \* CoLLM-DC/CoLLM-CC: 120
- Agent learning rate
  - \* MAGRPO:

- *StrBuild*:  $5 \times 10^{-6}$
- *HouseBuild*:  $1 \times 10^{-5}$
- \* CoLLM-DC/CoLLM-CC:
  - *StrBuild*:  $2.5 \times 10^{-6}$
  - *HouseBuild*:  $5 \times 10^{-6}$
- Critic learning rate (if applicable):
  - \* CoLLM-DC/CoLLM-CC:
    - *StrBuild*:  $1.5 \times 10^{-6}$
    - *HouseBuild*:  $3 \times 10^{-6}$
- Advantage clip: 0.05
- Number of evaluation samples: 2

## D. Additional Results

We provide additional results in our experiments.

### D.1. Pass@k Results on CoopHE

| Method       | Pass@1      | Pass@3      | Pass@5      | Pass@10     |
|--------------|-------------|-------------|-------------|-------------|
| Single-Agent | 56.3        | 58.7        | 61.9        | 62.5        |
| GRPO         | 61.8        | 62.0        | 62.2        | 62.8        |
| AC           | 62.5        | 62.6        | 63.0        | 63.3        |
| Parallel     | 50.0        | 51.1        | 68.8        | 68.8        |
| Pipeline     | 62.5        | <b>76.4</b> | 77.3        | 85.3        |
| Discussion   | 25.0        | 31.5        | 34.3        | 74.7        |
| MAGRPO       | 74.3        | 76.0        | 77.5        | 85.3        |
| CoLLM-DC     | 59.1        | 60.5        | 60.5        | 62.8        |
| CoLLM-CC     | <b>75.2</b> | 75.9        | <b>77.8</b> | <b>86.5</b> |

Table 3. Pass@k results (1, 3, 5, 10) on *CoopHE*. **Bolds** indicate the best performance. Results are averaged over 5 runs.

Table 3 presents the pass@k performance of coding collaboration on *CoopHE*. Fine-tuning with GRPO and AC yields marginal improvements over the raw model. However, this improvement over the given model is not due to acquired algorithmic knowledge or increased capacity. Instead, the training primarily refines the model’s policy, increasing the likelihood of producing correct solutions that already lie within its representational scope (Yue et al., 2025).

Most prompt-based multi-agent approaches underperform single-model baselines without proper optimization. Agents do not have timely communication to reason about each other’s correctness or functionalities. Although this challenge can be overcome through sequential execution, it reduces inference speed because agents take turns responding.

MARL methods can achieve performance comparable to or better than that of a single larger model. CoLLM-CC consistently achieves the best results across most pass@k results, because the main agent is guided to infer the auxiliary agent’s functionality and provide fallback solutions

when the auxiliary utilities are vulnerable, e.g., the main can provide a boundary condition manipulation. Also, through MARL optimization, the main completes the remaining components of the implementation, thereby accelerating inference through parallel execution. CoLLM-DC performs worse than CoLLM-CC and MAGRPO due to non-convergence (Fig. 2c).

### D.2. Training Overhead

We use the setting of *CoopHE* and *StrBuild* in Appendix C as a representative to compare MARL training overhead in Table 4 and Table 5.

| Metric    | MAGRPO | CoLLM-DC | CoLLM-CC |
|-----------|--------|----------|----------|
| #Epochs   | 8      | 80       | 80       |
| #Rollouts | 16     | 2        | 2        |
| #Samples  | 9640   | 8592     | 8438     |
| #Updates  | 603    | 2148     | 2110     |
| Duration  | 4.5    | 13.4     | 11.1     |
| VRAM      | 93.8   | 126.3    | 107.4    |

Table 4. Training overhead of MAGRPO, CoLLM-DC, and CoLLM-CC on *CoopHE*, under the settings of Appendix C. Metrics include the number of epochs, rollouts per episode, total samples used, policy updates, training duration (*H200* hours), and VRAM (GB) usage. Results are averaged over 5 runs.

Since MAGRPO is an instance of  $K$ -sampling, the rollout in an  $H$ -horizon episode forms a  $K$ -ary tree and satisfies Proposition 4.3. Under the hyperparameter setting in Appendix C, with  $K = 4$  and  $H = 2$ , MAGRPO produces  $K^H = 16$  rollouts and uses  $\sum_{k=1}^H K^k = 20$  samples, assuming no early termination is triggered. In contrast, both CoLLM-DC and CoLLM-CC generate a single rollout consisting of  $H = 2$  samples. To ensure a comparable number of training samples for MARL, we therefore train CoLLM-DC and CoLLM-CC for 10 times as many epochs as MAGRPO. Also, MAGRPO operates with a larger effective minibatch size. Accordingly, the agent’s learning rate is designed to scale proportionally, resulting in fewer updates.

Training CoLLM-DC and CoLLM-CC requires substantially longer time and more GPU memory than MAGRPO, as critic LLMs must be maintained throughout training. However, this overhead does not scale linearly with the number of LLMs in the systems ( $126.3 \ll 93.8 \times 2$ ). This is because the critic provides lower-variance gradient estimates, thereby reducing the number of samples that need to be retained in GPU memory for back-propagation. Furthermore, CoLLM-CC employs a single centralized critic, whose update cost is lower than maintaining  $n$  independent critics of the same size in CoLLM-DC, thereby reducing training time. As CoLLM-CC converges faster (Fig. 2c) and

a lot of samples trigger early termination during the early stages of training, it ultimately uses the fewest samples.

| Metric       | MAGRPO       | CoLLM-DC     | CoLLM-DC <sup>‡</sup> | CoLLM-CC     |
|--------------|--------------|--------------|-----------------------|--------------|
| <i>n</i> = 2 |              |              |                       |              |
| #Epochs      | 16           | 120          | 120                   | 120          |
| #Samples     | 6470         | 6264         | 6336                  | 5620         |
| Duration     | 48h (1×H200) | 52h (4×H100) | 18h (1×H200)          | 20h (1×H200) |
| VRAM         | 98G          | 188G         | 126G                  | 145G         |
| <i>n</i> = 3 |              |              |                       |              |
| #Epochs      | 6            | –            | 80                    | 80           |
| #Samples     | 3488         | –            | 6929                  | 5571         |
| Duration     | 68h (2×H200) | –            | 43h (2×H200)          | 46h (2×H200) |
| VRAM         | 167G         | –            | 219G                  | 263G         |

Table 5. Training overhead of MAGRPO, CoLLM-DC, CoLLM-DC with shared actor-critic parameters (CoLLM-DC<sup>‡</sup>), and CoLLM-CC on *StrBuild*, under the settings of Appendix C. Metrics include the number of training epochs, total samples used, training duration, and VRAM usage.

Table 5 demonstrates the training overhead as the number of agents increases. MAGRPO incurs rapidly increasing training time as the number of agents grows, making it difficult to scale to larger MAS. CoLLM-DC faces a similar issue, since each agent requires its own actor and critic. Although CoLLM-DC<sup>‡</sup> mitigates this overhead through parameter sharing, decentralized training remains susceptible to non-stationarity. CoLLM-CC avoids additional inference-time overhead under decentralized execution and achieves more stable training by using a centralized critic, but its training cost still increases with the MAS size. Developing more efficient and scalable training algorithms for multi-agent LLM systems remains an important future direction.

## E. Prompt Design Details

### E.1. Writing Collaboration

In the *TLDR* summarization, the instructions for each agent are as follows.

#### Summary Agent

Create a concise summary response to this post.  
 Query: {prompt}  
 Instructions: Provide a brief and focused summary in a few sentences

#### Elaboration Agent

Create a detailed summary response to this post.  
 Query: {prompt}  
 Instructions: You should use transition words to improve flow

In the *arXiv* expansion, we use the abstract field of the dataset and process it as follows.

#### Background Agent

Based on the following scientific abstract, expand the content for the introduction section.

Abstract: {abstract}

Instructions:

- There is another agent that will provide the method and implications
- You just need to focus on the background and motivation
- Avoid repeating methodology and implications content

#### Method Agent

Based on the following scientific abstract, expand the content for the introduction section.

Abstract: {abstract}

Instructions:

- There is another agent that will provide the background and motivation
- You just need to focus on the method and implications
- Avoid repeating background and motivation content

### E.2. Coding Collaboration

For CoopHE, we extract the `entry_point`, `params` from the prompt field and instruct the agents as follows.

#### Auxiliary Agent

Create a helper function for this coding problem.

Problem: {prompt}

Instructions:

- Output ONLY the function code, no explanations or examples
- Do NOT include markdown code blocks (````python`)
- Do NOT include any text before or after the function
- Do NOT include test cases or example usage
- Create a helper function named 'aux' that can assist the main function
- The function should return useful data for solving the problem

Your output should follow this format:

```
def aux(...):
    # your code here
    return result
```

#### Main Agent

Solve this coding problem by implementing the required function.

Problem: {prompt}

You have access to a helper function: `aux(...)`

Instructions:

- Output ONLY the function code, no explanations or examples
- Do NOT include markdown code blocks (``python)
- Do NOT include any text before or after the function
- Do NOT include test cases or example usage
- Do NOT redefine the aux() function
- Implement ONLY the '{entry\_point}' function as specified
- You can call aux() to assign a value to a variable within your function if helpful

Your output should follow this format:  
 def {entry\_point}({params}):  
 # your function code here  
 return result

To improve the generated code, agents receive additional feedback in addition to the initial problem description. This feedback comes from the static analyzer and sandbox tests, and is appended to the prompts for subsequent turns.

Static and execution diagnostics:  
 - Main definition: FOUND (Main function prime\_fib defined)  
 - Syntax: OK (Combined code syntax OK)  
 - Tests: 4/5 passed  
 assert candidate(1) == 2  
 AssertionError: expected 2, got 1

Revise your prime\_fib accordingly.

### E.3. Minecraft

In *StrBuild*, each agent is instructed by the following prompts at the first turn. The target specifications, available block textures, and building boundaries are provided as `target_ascii`, `block_agent_lines`, and `world_bbox_from` and `world_bbox_to` to the agent, respectively.

You are Player *i* in an *n*-person Minecraft building team. You will place SOME of the blocks for the final build. Output must be Minecraft commands only (no markdown, no code fences, no extra text).

The target is a character grid made of '#' and '.'. Its size matches the `bbox` and target rows. Output /setblock commands for a subset of '#' positions.

Target grid (top-down rows):  
 target\_ascii

WORLD bbox (inclusive):  
 - from: {world\_bbox\_from}  
 - to: {world\_bbox\_to}

Coordinate mapping (absolute coords):  
 - Let (min\_x, min\_y, min\_z) be `bbox.from` and (max\_x, max\_y, max\_z) be `bbox.to`.  
 - *x* increases from min\_x to max\_x (left to right).  
 - *y* increases from min\_y to max\_y (bottom to top).  
 - *z* is constant (min\_z == max\_z)

Available blocks (use ONLY these):  
 {block\_agent\_lines}

Constraints:  
 - Output ONLY Minecraft commands, one per line.  
 - Allowed commands: /setblock only.  
 - Use absolute integer coordinates only (no ~).  
 - Place blocks ONLY at '#' positions; leave '.' as air.  
 - Adjacent blocks (sharing a side) must NOT be the same texture.  
 - Every coordinate must be within the `bbox`.

Format: /setblock <x> <y> <z> <block>

In *HouseBuild*, since the player can construct blocks flexibly at any adjacent location, the positions of players are assumed to be the center. We employ a simulator to generate `spider_num` spiders from arbitrary locations on the map. It also computes the player's health points `player_hp`, and the damage inflicted on spiders `spider_dmg`. The predefined player attack value is `player_atk`, while each spider is assumed to have an attack value `spider_atk`.

You are Player *i* in an *n*-person Minecraft building team. You will place SOME of the blocks for the final build. Output must be Minecraft commands only (no markdown, no code fences, no extra text).

Task: Build the 3D structure from the provided *y*-axis slices.

Available blocks (use ONLY these):  
 {block\_agent\_lines}

Layers (ascending WORLD *y*). Each layer is a set of rectangles in WORLD (*x*, *z*) coords:  
 - Format: {(x1, y1, z1, x2, y2, z2 block\_id), (x1, y1, z1, x2, y2, z2 block\_id)}

WORLD bbox (inclusive):  
 - from: {world\_bbox\_from}  
 - to: {world\_bbox\_to}

Threat check: There are {spider\_num} spiders nearby; you have {player\_hp}

```

    HP. Spider attack list: {spider_atk
    }, total damage dmg={spider_dmg}. If
    you want to kill them, output the
    attack command: /damage @e[type=
    spider,limit=1] {player_atk}
    minecraft:player attack. All other
    commands should be normal building
    commands.
    
```

Constraints:

- Output ONLY Minecraft commands, one per line.
- Allowed commands: /fill and /kill
- Fill format: /fill x1 y1 z1 x2 y2 z2 block
- Use absolute integer coordinates only (no ~).
- Use ONLY blocks from the legend.
- Every coordinate must be within the bbox.

And we provide DC and CC with the following global information,  $m_t$ , during training.

```

    You are at turn = {turn} over a {
    num_turns}-turn episode.
    Your last-turn reward is {last_reward} /
    {reward_max}.
    
```

## F. Reward Design Details

Reinforcement Learning with Verifiable Rewards (RLVR) provides objective and reliable training signals. However, most RLVR methods in a multi-turn setting rely on terminal rewards, resulting in extremely sparse feedback that complicates credit assignment across multiple agents. Recent work addresses this by introducing process-based or intermediate rewards, often instantiated as manually designed rubrics (Uesato et al., 2022; Lifshitz et al., 2025; Wu et al., 2025b). However, these approaches typically require substantial manual intervention for rubric engineering. It remains unclear whether such designs are still generalizable, consistent, and aligned with the intended objectives.

We provide verifiable reward signals at each turn, enabling fine-grained and objective supervision throughout the interaction. However, the cumulative return becomes dependent on the horizon, resulting in a dynamic return scale that must be carefully handled in policy optimization. We solve this problem by providing the critic with global information  $m_t$ , as shown in critic prompts in Appendix E.

### F.1. Writing Collaboration

We evaluate *TLDR* summarization along three dimensions: structural quality, style consistency, and logical coherence. Structural wellness measures the relative completion length and lexical diversity (unique-word ratio excluding stop-words). Full rewards are assigned when the length ratio lies

in  $1.6\text{--}3.2\times$ , and the unique-word ratio is  $\geq 2.0\times$ ; values within broader tolerances ( $1.1\text{--}5.0\times$  for length and  $1.3\text{--}2.0\times$  for uniqueness) receive proportionally scaled rewards, while out-of-range cases incur zero reward and early termination. Style consistency is quantified by the Jaccard similarity of vocabularies between completions (excluding stopwords), capped at 0.03 to balance lexical consistency with vocabulary expansion. Logical coherence is evaluated through transition-word usage in the combined text across 12 functional categories, with rewards scaled by category diversity as  $\min(0.6 \log(\#\text{categories} + 1), 1)$ .

For arXiv expansion, we assess structural wellness, style consistency, and logical coherence by comparing the second paragraph against the first. Structural wellness rewards optimal length ratios of  $1.0\text{--}1.3\times$  and unique-word ratios of  $0.7\text{--}1.3\times$ , assigns proportional rewards within acceptable ranges ( $0.8\text{--}1.5\times$  for length and  $0.5\text{--}1.7\times$  for uniqueness), and terminates evaluation otherwise. Style consistency is measured using Jaccard similarity between the two completions, capped at 0.23 and normalized as the reward. Logical coherence is evaluated through transition-word usage across the same 12 categories, with rewards scaled by category diversity as  $\min(0.4 \log(\#\text{categories} + 1), 1)$ .

### F.2. Coding Collaboration

We evaluate coding collaboration along four dimensions: structural integrity, syntactic correctness, test pass rate, and cooperation quality. Structural integrity verifies that both auxiliary and main functions are properly defined, with valid signatures and return statements; failure to define the main function results in immediate termination. Syntactic correctness assesses whether the concatenated code parses without errors, verified via the Abstract Syntax Tree (AST) library. Any syntax error leads to evaluation termination to prevent runtime failures. The test pass rate measures the proportion of unit tests that successfully pass within an 8-second timeout, with rewards scaled by the number of successful assertions ( $\text{tests}$ )  $\frac{\#\text{passed\_tests}}{\#\text{total\_tests}}$ , and termination triggered if no tests pass. Cooperation quality assigns a base bonus when the main function invokes the auxiliary, with additional rewards granted when the main function exhibits substantive logic beyond a simple wrapper. A deduction is applied if the main function calls the auxiliary but discards its return value, penalizing superficial cooperation.

### F.3. Minecraft

In *StrBuild*, we first design the reward to encourage agents to construct string-like structures in accordance with the given paradigms using coverage rate  $\frac{2\#\text{covered\_blocks}}{\#\text{total}}$ . The agents are also optimized to minimize unnecessary resource consumption  $-\frac{1.5\#\text{extra\_blocks}}{\#\text{total}}$ . In addition, we introduce an adjacency penalty based on the same-texture adjacency

rate,  $\frac{\# \text{same\_texture\_pairs}}{2 \# \text{total\_pairs}}$ , which discourages excessive adjacency of identical block textures. In HouseBuild, we similarly employ the coverage rate and redundancy rate to encourage accurate and efficient construction. Also, agents are penalized for damage inflicted by spiders as  $\min\left(1, \frac{\text{spider\_total\_dmg}}{\text{player\_hp}}\right) \times 0.2$ .

## G. Compute Resources

The compute resources used in our experiments (Sec. 6) for training and evaluation are listed below.

### G.1. Training Devices

- **Writing Collaboration**
  - Type: GPU Cluster  
CPU: NVIDIA Grace ARMv9  
GPU: 1× NVIDIA Hopper H100
- **Coding Collaboration**
  - Type: GPU Cluster  
CPU: Intel Xeon Platinum 8558  
GPU: 1× NVIDIA Hopper H200
  - Type: GPU Cluster  
CPU: Intel Xeon Gold 5318Y  
GPU: 1× NVIDIA Hopper H200
  - Type: GPU Cloud Instance  
CPU: AMD EPYC 9755  
GPU: 1× NVIDIA Hopper H200
  - Type: GPU Cloud Instance  
CPU: Intel Xeon Platinum 8568Y+  
GPU: 1× NVIDIA Hopper H200
- **Minecraft Building Collaboration**
  - Type: GPU Cluster  
CPU: Intel Xeon Platinum 8592+  
GPU: 1× NVIDIA Blackwell B200
  - Type: GPU Cluster  
CPU: Intel Xeon Platinum 8558  
GPU: 1× NVIDIA Hopper H200
  - Type: GPU Cloud Instance  
CPU: Intel Xeon Platinum 8568Y+  
GPU: 1× NVIDIA Blackwell B200
  - Type: GPU Cloud Instance  
CPU: AMD EPYC 9655  
GPU: 1× NVIDIA Hopper H200
  - Type: GPU Cloud Instance  
CPU: AMD EPYC 9755  
GPU: 1× NVIDIA Hopper H200
  - Type: GPU Cloud Instance  
CPU: Intel Xeon Platinum 8592+  
GPU: 1× NVIDIA Hopper H200

### G.2. Inference Device

- **All Tasks**
  - Type: Standalone Workstation  
CPU: AMD Ryzen 9 9950X  
GPU: 1× NVIDIA GeForce RTX 5090

## H. Code and Dataset

Our code and dataset are available as follows.

- **CoMLRL: Cooperative Multi-LLM Reinforcement Learning** (CoMLRL) is an open-source library for training multiple LLMs to collaborate using Multi-Agent Reinforcement Learning (MARL). It provides implementations of various MARL trainers for decentralized LLM collaboration, <https://github.com/OpenMLRL/CoMLRL/releases/tag/v1.3.6>. Please refer to the documentation at <https://github.com/OpenMLRL/CoMLRL> for more detailed usage examples.
- **LLM\_Collab\_Writing**: This repository contains our writing collaboration experiments, [https://github.com/OpenMLRL/LLM\\_Collab\\_Writing/releases/tag/v1.3.6](https://github.com/OpenMLRL/LLM_Collab_Writing/releases/tag/v1.3.6).
- **LLM\_Collab\_Code\_Generation**: This repository contains coding collaboration experiments, [https://github.com/OpenMLRL/LLM\\_Collab\\_Code\\_Generation/releases/tag/v1.3.6](https://github.com/OpenMLRL/LLM_Collab_Code_Generation/releases/tag/v1.3.6).
- **LLM\_Collab\_Minecraft**: This repository contains Minecraft game-playing experiments, [https://github.com/OpenMLRL/LLM\\_Collab\\_Minecraft/releases/tag/v1.3.6](https://github.com/OpenMLRL/LLM_Collab_Minecraft/releases/tag/v1.3.6).
- **CoopHumanEval**: Our CoopHumanEval dataset we used is open-sourced at <https://huggingface.co/datasets/OpenMLRL/CoopHumanEval>.